CFG NINJA

SECURITY ASSESSMENT

# Chain Core, Consensus & Post-Quantum Cryptography

**FINAL REPORT**

Client: Krown Network
Network: Krown Network mainnet · Chain ID 1983 (0x7BF)
Codebase: github.com/Krown-Network/Node @ core-block
Commit: dd105cbf6d7ab0ceeba955fe489f92701bd430bd
Assessment date: 31 July – 3 August 2026
Report generated: 2026-08-04 13:32 UTC
Prepared by: CFG Ninja — Security Assessment Team

**PQR-2**
POST-QUANTUM READINESS
Experimental

# Report Status, Scope & Limitations

This document is CFG Ninja's FINAL report on the chain-core, consensus and post-quantum assessment of Krown Network, prepared for Krown Network as its client. Krown commissioned it to support Krown's own security due diligence and intends to provide it to BloFin as part of the exchange's listing review. It reflects a source-level review of the core-block integration branch of github.com/Krown-Network/Node at commit dd105cbf6d7ab0ceeba955fe489f92701bd430bd — confirmed to be the live mainnet build — together with a mechanical fork diff against upstream go-ethereum v1.16.2, independent verification of the client's round-1 remediation branch, live-network verification, and cross-referencing of the client's pre-audit briefing, its due-diligence responses to BloFin, and production artifacts. The assessment is complete within the agreed scope; the verdict is a conditional pass whose conditions are set out in section 12, and it is not effective until those conditions are met and re-verified.

## Engagement & relationship

Engagement relationship. CFG Ninja was engaged by Krown Network to perform this assessment. Krown is the client and commissioned the work to satisfy its own security due-diligence obligations for the chain core, consensus and post-quantum implementation. Krown intends to provide this report to BloFin as part of the exchange's listing review. BloFin did not commission this assessment and CFG Ninja has no contractual relationship with BloFin.

CFG Ninja's role in this arrangement is that of an independent third-party assessor. The value of the report to both Krown and BloFin rests entirely on that independence: CFG Ninja provides an external, reproducible and transparent examination of the implementation, so that neither party has to take the other's representations on trust. A report the client could shape would provide neither transparency nor validity and would be worthless to the exchange; accordingly the client commissioned the work but does not control its conclusions.

That the client both commissioned the report and will deliver it to the exchange is precisely why the independence terms below matter, and why they are stated rather than assumed. The report is written to be relied upon by BloFin and by any third party performing diligence on Krown, not only by the client who paid for it.

## What this assessment covers

- Static source-level review of the core-block integration branch at commit dd105cbf, plus comparative review of all 8 remote branches (main, dev, dev-test, staging, core-block, dilit-multi-sig, docker-k8s-config, update-config).
- Upstream baseline identification and enumeration of the modification surface against stock go-ethereum v1.16.2.
- Full post-quantum cryptography review: scheme, parameter set, library provenance, enforcement layer, and address-derivation impact for both Dilithium (signatures) and Kyber (RLPx transport).
- Custom transaction type 0x75: encoding, sender recovery, address derivation and mempool admission.
- Committed-secret scan across all 8 branches and the complete 169-commit history.
- Consensus parameters, genesis and fork-schedule configuration versus canonical Ethereum mainnet presets; consensus-client identity reconciliation.
- Deployment manifests (Docker Compose, Kubernetes, Kurtosis) and node-operations configuration.
- Cross-reference of implemented reality against the client pre-audit briefing and the BloFin due-diligence responses.

- Live-network verification against the client's declared public RPC endpoint: chain ID, client build, sync state, block height, finality tag support, live gas limit, and JSON-RPC namespace exposure.
- Static analysis with gosec and staticcheck, dependency-vulnerability analysis with govulncheck, and measured test coverage over the modification surface — all executed, with results triaged rather than reported raw.
- Reconciliation of the two client documents (the BloFin due-diligence responses and the Krown pre-audit technical briefing) against each other and against source.
- Mechanical fork diff against a vendored upstream go-ethereum v1.16.2, confirming the modification surface and that all consensus/state/sync paths are stock (CORE-CFG02).
- Independent verification of the client's round-1 remediation branch (remediation/cfg-round1): the diffs were checked directly and govulncheck re-run, not accepted on the commit hashes.

## Conditions and items carried forward

- DEPLOYMENT of the round-1 fixes to the production fleet. The fixes are complete and CFG Ninja-verified in source, but the live fleet still runs commit dd105cbf; until the rebuilt image is rolled out the fixes are not in effect on the live network (CORE-CFG01, CORE-CFG06). CFG Ninja will issue a short deployment-verification addendum once it confirms the live build, at which point the near-term conditions close.
- Confirmation that the corrected due-diligence text has been delivered to BloFin (CORE-CFG18 — wording already verified accurate against source).
- Stage-gated remediations, disclosed and on the client's pre-external-operator roadmap: CORE-CFG08 (Eth1 follow distance), CORE-CFG24 (handshake proof-of-possession), and CORE-CFG14/CFG16 (consensus-level post-quantum enforcement and hybrid KEM).
- One open low-severity finding, CORE-CFG09 (checkpoint-sync trust), to be addressed before external operators.
- State management and sync runtime behaviour was not independently exercised (no runnable node); the mechanical diff confirms those paths are stock upstream, so residual risk is bounded and disclosed as a scope limitation.

## Independence & prior engagement

CFG Ninja was engaged and paid by Krown Network, the client. Independence was maintained notwithstanding: no finding in this report was supplied by the client, the client had no ability to alter or suppress a finding, and every finding is cited to file and line on the reviewed commit so that BloFin or any third party may reproduce it rather than take it on trust. Where the client's own briefing, its documentation, or its due-diligence responses to BloFin disagree with the code, this report follows the code and says so explicitly — several such contradictions are recorded in these findings, and in each case the report sides with the evidence over the client. CFG Ninja also corrected two of its own errors during the engagement rather than conceal them (a withdrawn systemic-risk item and a mis-scoped chain-ID finding), which is recorded for the same reason: a report a client hands to an exchange is only worth handing over if it is demonstrably not the client's to shape.

CFG Ninja previously performed a security assessment of the kDex decentralised-exchange platform for the same client group. That engagement covered application and smart-contract scope only and shares no code, no findings and no personnel dependency with this chain-core review.

# 1. Executive Summary

Krown Network is a live production Layer 1. Mainnet launched on 3 January 2026, runs chain ID 1983, and had produced roughly 1.47 million blocks at the time of this assessment. It is a fork of go-ethereum v1.16.2 extended with a post-quantum layer: a Kyber key-encapsulation mechanism in the RLPx peer-to-peer handshake, a CRYSTALS-Dilithium signing package, a Dilithium-attested RPC submission endpoint, and a custom transaction type (0x75) using SHAKE256 for its signature hash and sender-address derivation.

One verification frames everything that follows. The live mainnet node reports its build as Geth/v1.16.2-stable-dd105cbf — commit dd105cbf, the exact commit reviewed throughout this report. Every source-level finding here therefore describes code running on Krown mainnet today. The usual audit caveat about whether the reviewed tree matches production does not apply.

The post-quantum transport layer is real and mandatory: every RLPx connection performs a Kyber768 encapsulation and session keys derive from the Kyber shared secret. The post-quantum signature layer is not. Dilithium signatures are verified at the RPC boundary and then discarded; no consensus rule, state-transition path, block validator or transaction-pool check anywhere in the codebase requires a post-quantum signature for any account or transaction type. The trust root for every transaction on the chain, including type 0x75, is classical secp256k1 ECDSA. Krown's own pre-audit briefing states this accurately and explains the intended design: enforcement is meant to occur at the wallet layer. That is a coherent product architecture, but it is not a chain-level control, because a wallet-layer check binds only those who use the wallet and the chain accepts any valid ECDSA transaction submitted directly. An adversary who has broken ECDSA is precisely the adversary who will not use the wallet.

That gap matters here more than it normally would, because the due-diligence response sent to BloFin states the opposite of what the code and the client's own internal briefing both say. Where the two client documents diverge — on transaction signatures, on QRNG entropy, on NIST standardisation; the internal briefing is accurate and the document sent to the exchange overstates. A reader of the BloFin response alone would conclude the chain has post-quantum transaction signatures, NIST-standardised primitives and quantum entropy in production. It has none of those three. This is the single most important item in this report for a listing decision, and correcting it is straightforward.

Three findings were originally rated High: the derived session secret written to standard output on every handshake (CORE-CFG06); the discarded result of the peer identity-signature check (CORE-CFG24); and the unenforceable post-quantum signature layer (CORE-CFG14). None remains open at High. CORE-CFG06 is fixed and CFG Ninja-verified in source, with deployment to the live fleet pending. CORE-CFG14 is scored at Medium, its representation aspect tracked and now corrected at CORE-CFG18. CORE-CFG24 is reported scope-limited at Medium: the source defect is confirmed and in scope, while its exploitable severity turns on network exposure that lies outside the agreed chain-core scope and is deferred to a network assessment. CORE-CFG06 and CORE-CFG24 both escalate at the external-validator milestone and are being remediated ahead of it. No Critical or High finding is open. The Medium findings cover pre-standard cryptographic parameter sets, a dual-address derivation hazard, an unreachable transaction type, a dependency-vulnerability set since reduced from seven to two, and the documentation gaps above; test coverage of the custom code is recorded as out of scope.

No Critical finding is confirmed. Nothing indicates malice, a backdoor or a hidden mint. Several results are genuinely good and are recorded as such: consensus parameters are unmodified mainnet presets with no weakening of slashing or rewards; the public RPC surface is correctly hardened with admin, debug, personal and miner namespaces all disabled; the finalized and safe block tags resolve, so BloFin's recommended deposit-crediting model is sound; the post-quantum primitives come from a reputable library rather than hand-rolled

cryptography; and the client's own secret-derivation work was methodologically sound as far as it went. What the review shows overall is a chain whose engineering is more capable than its documentation is careful, carrying an assurance gap; no tests on the rewritten security-critical code, no analyser in CI; that explains how most of these findings survived to production.

## Headline assessment

- The audited commit is the production build. Live mainnet reports Geth/v1.16.2-stable-dd105cbf, matching the reviewed core-block HEAD exactly — every finding below describes code running on Krown mainnet today.
- The two client documents contradict each other, and the one sent to BloFin is the inaccurate one. Krown's internal briefing correctly states that consensus authentication rests on ECDSA; the BloFin response states that signature verification uses Dilithium. (CORE-CFG18)
- The post-quantum SIGNATURE layer is not enforceable. SendRawTransactionWithDilithium verifies a Dilithium signature and then submits the plain ECDSA transaction, discarding the Dilithium material. The identical transaction can be submitted through standard eth_sendRawTransaction with no Dilithium data and is accepted identically. (CORE-CFG14)
- The post-quantum TRANSPORT layer is genuine and mandatory. Kyber768 is wired into every RLPx handshake with no downgrade path and no capability negotiation. (CORE-CFG16)
- Both post-quantum primitives are PRE-STANDARD round-3 schemes, not the finalised NIST standards: CIRCL sign/dilithium/mode2 rather than FIPS-204 ML-DSA, and CIRCL kem/kyber/kyber768 rather than FIPS-203 ML-KEM. The repository contains no ML-DSA or ML-KEM code at all. (CORE-CFG13, CORE-CFG16)
- The derived Kyber session secret is printed to standard output on every handshake, immediately before being zeroed. (CORE-CFG06)
- The RLPx handshake recovers the peer's identity public key and then discards the result, removing the check that a connecting peer controls the key for the identity it claims. (CORE-CFG24)
- The repository contains no production chain configuration. Mainnet genuinely runs chain ID 1983 (verified live, and registered as eip155:1983), but every manifest in the audited repository configures the stock Kurtosis devnet default 3151908. The execution-client source is production; the configuration beside it is not. (CORE-CFG12)
- Every deployment artefact on every branch configures Lighthouse as both beacon node and validator client. The Prysm v7.1.3 claim in the pre-audit briefing is not substantiated anywhere in the repository. The BloFin due-diligence response is the accurate one. (CORE-CFG07)
- Seven distinct mnemonics are committed to the repository — one more than the six disclosed in the briefing. Six are recognisable public test vectors or obvious placeholders; one, on the update-config branch, is unattributed and must be cleared by offline derivation before it can be dismissed. (CORE-CFG19)
- The custom code is effectively untested. Of 321 test files in the tree, essentially all are inherited from upstream; crypto/kyber, p2p/rlpx and internal/ethapi have none, and the upstream tests that covered the rewritten handshake are absent. Neither CI workflow runs the test suite. This is the direct explanation for how CORE-CFG24 and CORE-CFG04 shipped undetected. (CORE-CFG34)
- Consensus parameters are NOT weakened. Slashing quotients, inactivity penalties, effective balance, ejection balance, churn limit and slot duration are all inherited unmodified from the mainnet preset. This is a clean result. (CORE-CFG08)
- No open Critical or High severity finding remains. CFG Ninja's final determination is a CONDITIONAL PASS, subject to deployment of the verified round-1 fixes to the live fleet and the stage-gated roadmap items — it is not a pass of the current live code. The conditions are in section 12.

## Vulnerability Summary

| 24 | 10 | 9 | 1 |
|---|---|---|---|
| Total Findings | Resolved | Acknowledged | Open |

Assessed against the CFG Ninja CORE-CFG01–CFG38 chain-core framework, with govulncheck, gosec, staticcheck and measured coverage over the modification surface.

The 1 open item is low-severity CORE-CFG09. Separately, 3 informational observations (scored zero) and 1 out of scope require no remediation.

| Severity | Resolution Status | Description |
|---|---|---|
| 0 Critical | – | Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise. |
| 0 High | – | Significant risk that should be addressed urgently to prevent exploitation or financial loss. |
| 13 Medium | 7 resolved · 5 acknowledged · 1 out of scope | Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities. |
| 6 Low | 1 resolved · 4 acknowledged · 1 open | Minor security concern. Best practice violations or low-impact issues. |
| 5 Informational | 2 resolved · 3 informational | Code quality or optimization suggestions with no direct security impact. |

**CONDITIONAL PASS**

No open Critical or High severity finding. Full verdict, conditions for a passing final report, and the code-maturity matrix are in section 12.

# 2. Network Reference Card

Chain parameters as represented by the client, retained here for the integration team and reconciled against source where possible.

| | |
|---|---|
| Chain / project | Krown Network |
| Status | Production mainnet, live since 3 January 2026. ~1,469,506 blocks at time of check (1 August 2026). eth_syncing: false. |
| Public RPC | https://mainnet.krown.network (verified live). Second endpoint mainnet1.krown.network per client. Explorer: explorer.krown.network (Blockscout). |
| Finality tags | finalized and safe both resolve — verified live. Client recommends BloFin credit deposits on the finalized tag; that recommendation is sound. |
| Execution client | Geth/v1.16.2-stable-dd105cbf/linux-amd64/go1.24.11 — read live from mainnet. Build commit matches the reviewed source exactly. |
| Consensus client | Lighthouse (sigp/lighthouse:latest): beacon node and validator client, on every branch. NOT Prysm v7.1.3 as stated in the pre-audit briefing. Image tag is floating :latest, not pinned. |
| Consensus mechanism | Ethereum Proof-of-Stake: LMD-GHOST fork choice with Casper FFG finality, inherited unmodified from upstream. |
| Chain ID | 1983 (0x7BF): verified live via eth_chainId. Registered in ethereum-lists/chains as eip155:1983. |
| Chain ID in the repository | 3151908 on core-block/docker-k8s-config; 89127398 on update-config. Neither is the production value — the repository holds devnet configuration only. See CORE-CFG12. |
| Genesis gas limit | 60,000,000 (core-block); 45,000,000 (update-config). Approximately 2x typical Ethereum mainnet. |
| Fork schedule | Stock upstream mainnet fork table (Homestead through Prague timestamps), unmodified. No Krown-specific chain-config object exists in params/config.go. |
| Beacon preset | PRESET_BASE=mainnet. Validator balance 32 ETH, ejection 16 ETH, churn-limit quotient 65536, slot duration 12s, withdrawability delay 256, shard-committee period 256 — all matching mainnet. |
| PQC: transport | CRYSTALS-Kyber768 KEM (round-3), mandatory in every RLPx handshake, non-hybrid. CIRCL v1.6.1. |
| PQC: signatures | CRYSTALS-Dilithium mode2 (round-3), CIRCL v1.6.1 — RPC-layer attestation only, not consensus-enforced. |
| PQC: hashing | SHAKE256, confined to transaction type 0x75 sig-hash and its sender-address derivation. All other hashing remains Keccak-256. |
| Custom tx type | 0x75 (PQCTxType): registered as an EIP-2718 typed envelope; not admissible via the standard transaction pool (CORE-CFG04). |
| Repository | github.com/Krown-Network/Node — 8 remote branches, 169 commits, squashed re-import history. |
| Reviewed commit | dd105cbf6d7ab0ceeba955fe489f92701bd430bd (core-block) |

| | |
|---|---|
| Deployment tooling | Docker Compose, Kubernetes (Helm-style values.yaml + StatefulSets), and Kurtosis ethereum-package. |
| Validator set | Fully permissioned, Krown-operated at time of review. Open/external validation is a stated roadmap milestone. |

# 3. Scope & Methodology

## Repository & branches reviewed

- core-block @ dd105cb — INTEGRATION TRUNK and the primary subject of this review. Carries the full PQC feature set including crypto/dilithium and the Dilithium RPC endpoint.
- origin/main @ f494a1f — plain client source tree; contains no devnet tooling directories.
- origin/dev @ 92ca8ce — file tree and secret footprint identical to core-block.
- origin/dev-test @ f302c76 — plain client source tree, no devnet tooling.
- origin/staging @ 204be60 — plain client source tree, no devnet tooling.
- origin/dilit-multi-sig @ 3acb2df — no differences from core-block on diff.
- origin/docker-k8s-config @ 92ef598 — near-identical to core-block; lacks crypto/dilithium and the 70-line Dilithium RPC block.
- origin/update-config @ 4f6430b — superset carrying a legacy configurations/ directory absent elsewhere; the crypto/dilithium package (256 lines) is DELETED on this branch, indicating the PQC work is in flux across branches. Carries additional committed mnemonics and a weakened Eth1 follow distance.

## In scope

- Execution-client fork integrity: modification surface versus stock go-ethereum v1.16.2, consensus-critical packages, transaction types, mempool admission, EVM and state-transition paths.
- Post-quantum cryptography: scheme identification, parameter sets, library provenance, enforcement layer, hashing and address derivation, transport KEM integration.
- Peer-to-peer transport: RLPx handshake construction, authentication, session-key derivation and secret handling.
- Consensus configuration: genesis, chain config, fork schedule, beacon preset parameters, consensus-client identity.
- Key and secret management: committed credentials across all branches and full history, .gitignore coverage, key-separation posture.
- Node operations: Docker/Kubernetes/Kurtosis manifests, RPC exposure, deployment-manifest integrity.
- Accuracy of the client's public and due-diligence representations against the implemented code.
- Engineering assurance of the reviewed packages: test coverage measured over the modification surface, retention of upstream tests through the fork, and the presence of automated test, static-analysis, dependency-scanning and secret-scanning gates in CI. Scope note: this category (CORE-CFG34, CORE-CFG38) was added to the CFG Ninja chain-core framework on 1 August 2026, during this engagement, and was therefore not named in the scope agreed at intake. It is declared here for transparency. The client has not accepted it as in scope and is not treated as owing a remediation response on it — see the scope query in the accompanying response assessment.

## Out of scope (this engagement)

- Smart contracts deployed on the network, including any token, bridge or staking contract. This is a chain-core assessment, not a smart-contract audit.
- The kDex decentralised-exchange application and its contracts, covered under a separate prior engagement.
- Genesis hash verification, genesis allocation and supply integrity — deferred to the final report. Correction: an earlier draft of this list excluded live-network state generally. That was inconsistent with the work actually performed — read-only live verification of chain identity, client build, sync state, finality tags, gas limit and RPC namespace exposure was carried out and is reported in section 6. Only the genesis and supply items remain deferred, and on-chain balance analysis was performed by the client and independently sampled by CFG Ninja in support of CORE-CFG04 and CORE-CFG15.
- Third-party bridge or cross-chain infrastructure and any audits thereof.
- Economic, tokenomic and governance design; legal, regulatory and custody classification.
- Physical, cloud-account and personnel security of the operating entity.
- Formal verification and cryptanalysis of the CIRCL primitives themselves; this review assesses their selection, parameterisation and integration, not their internal correctness.

## Method & tooling

- Direct source reading of all consensus-critical, cryptographic and peer-to-peer packages on the reviewed commit, with every finding cited to file and line.
- Comparative branch analysis across all 8 remote branches to identify feature drift and to establish which artefacts are trunk versus legacy.
- Signal-driven enumeration (targeted search for PQC, Dilithium, Kyber, SHAKE256, PQCTxType, 0x75 and related identifiers) across the whole tree, including negative searches to establish where a primitive is NOT used — the basis for the enforcement findings.
- Secret scanning with git grep and git log --all -G/-p across all 8 branch refs and all 169 reachable commits, covering mnemonics, keys, keystores, passwords, JWTs and tokens.
- Reconciliation of the client pre-audit briefing and the BloFin due-diligence responses against the implemented code, with disagreements resolved in favour of the code and reported as findings.
- Adversarial cross-verification: the post-quantum findings were independently re-derived by a second reviewer working from source, and are reported here only where both passes agreed.
- All repository access was read-only. No commits, pushes, branch mutations or writes of any kind were made to the client's repository.
- Standards alignment, stated precisely: this assessment corresponds to the review techniques described in NIST SP 800-115 (documentation review, ruleset and configuration review, and source-code review) and to the intelligence-gathering and analysis phases of the Penetration Testing Execution Standard. It does NOT include the target-validation techniques those standards also describe — no dynamic testing, no fuzzing, no proof-of-concept exploitation and no live-network testing were performed. Industry practice for a full Layer-1 assessment includes those techniques and attaches a proof-of-concept to every High and Critical finding; this assessment does not, and the the High findings here are therefore reported at source level with their verification steps listed in section 11. The final report closes that gap.
- Limitation stated plainly: no mechanical diff against a vendored upstream v1.16.2 tree was performed (see CORE-CFG02), and no code was compiled or executed. Findings are source-level; where a conclusion is inference rather than demonstration, the finding says so.

- AI-assisted analysis was used in this engagement, under human review. The methods, the oversight applied, the evidence-grading standard and the data-handling terms are disclosed in full in Appendix C — no finding here rests on an unverified automated assertion.
- Scope corrections applied 2026-08-02, recorded rather than silently amended: engineering assurance was assessed but had not been declared in scope, the category having been added to the framework during this engagement; and an earlier exclusion of live-network state contradicted the live verification actually performed and reported in section 6. Both are self-audit findings and are treated the same way as the withdrawn replay-exposure item.

# 4. CFG Ninja Post-Quantum Readiness Grade

**PQR-2**
Experimental

PQC present and executing, but not enforced: verified-then-dropped, optional endpoint, or confined to a non-consensus path that a standard path bypasses.

Krown Network grades PQR-2 (Experimental) on the CFG Ninja Post-Quantum Readiness scale. The grade recognises that the transport layer is genuinely post-quantum — Kyber768 is mandatory on every RLPx handshake, with no negotiation and no downgrade path, which is more than most chains claiming post-quantum status have built. It is held at 2 rather than 3 by four caps, every one of which is a bounded engineering fix rather than a redesign. The signature layer, which is what protects on-chain value, grades PQR-1: the capability is claimed but not enforced by any consensus rule.

## Caps applied

A chain receives the lowest grade any cap allows. Each cap below is cited to the finding that establishes it.

- Pre-standard primitives (caps at PQR-3): CIRCL round-3 kyber768 and dilithium/mode2 rather than FIPS-203 ML-KEM and FIPS-204 ML-DSA. The standardised implementations ship in the same pinned CIRCL version and are unused. See CORE-CFG13, CORE-CFG16.
- Non-hybrid construction (caps at PQR-3): the classical ECDH secret is computed but excluded from key derivation, so a flaw in the Kyber implementation has no classical fallback. See CORE-CFG16.
- Secret disclosure in the PQC path (caps at PQR-2): the derived Kyber session secret is written to stdout on every handshake at p2p/rlpx/rlpx.go:533. See CORE-CFG06.
- Representation gap (caps at PQR-2): published and due-diligence claims materially exceed the implementation, most significantly the statement that signature verification uses Dilithium. See CORE-CFG18.

## Per-layer grades

| Layer | Grade | Basis |
|---|---|---|
| Transport (KEM) | PQR-2 | Mandatory Kyber768 on every handshake, no downgrade path — PQR-3 behaviour, capped to 2 by pre-standard parameters, the non-hybrid derivation and the session-secret disclosure. The strongest layer, and the cheapest to raise. |
| Transaction signatures | PQR-1 | Claimed, not enforced. Dilithium is verified at the RPC boundary then discarded; every transaction including type 0x75 is authenticated by secp256k1 ECDSA. No consensus rule requires a post-quantum signature for any account. See CORE-CFG14. |
| Hashing & address derivation | PQR-1 | SHAKE256 is used for one transaction type's signing digest and its address derivation only. A different hash confers no post-quantum property on the signature over it; the change also creates the dual-address hazard in CORE-CFG15. Block, state and receipt hashing remain Keccak-256. |
| Validator / consensus keys | PQR-0 | Standard Ethereum BLS12-381 validator keys, inherited unmodified. Not quantum-resistant and outside the scope of the fork's post-quantum work. |
| Key management & rotation | PQR-0 | No post-quantum key lifecycle exists: no PQ key generation, custody, backup or rotation policy was found. Committed test credentials are addressed separately in CORE-CFG19. |

## Harvest-now-decrypt-later exposure: HIGH

Harvest-now-decrypt-later exposure is assessed separately from the grade because it is often the more actionable output, and here the two point in different directions.

Recorded-traffic exposure is MODERATE. An adversary recording Krown P2P traffic today faces a Kyber768-protected session, which is genuine protection against a future quantum adversary — this is the real benefit of the transport work. Two things qualify it: the construction is non-hybrid, so a flaw in the pre-standard Kyber implementation removes the protection entirely with nothing classical behind it; and more immediately, the session secret is written to stdout on every handshake, so any retained node logs from a deployment running this code contain the material needed to decrypt the corresponding recorded sessions today, with no quantum computer required.

Long-lived signature exposure is HIGH, and this drives the overall verdict. Every unit of value on the chain is secured by secp256k1 ECDSA. Unlike encryption, signatures cannot be retroactively strengthened: a public key exposed on-chain today is forgeable by whoever first holds a cryptographically-relevant quantum computer, and every address that has ever sent a transaction has exposed its public key. Custody, vesting and governance keys intended to remain valid for years are the concentrated form of this risk. The Dilithium endpoint does not mitigate it, because the signature it verifies is discarded and never becomes part of the authenticated state.

Overall HNDL: HIGH. The post-quantum work to date protects data in motion and leaves the value at rest classical.

## Migration path — next grade: PQR-3 (Transport-hardened)

Four bounded changes move Krown from PQR-2 to PQR-3, and none is a redesign:

1. Swap the KEM to CIRCL kem/mlkem/mlkem768 (FIPS-203 ML-KEM-768). It ships in the CIRCL version already pinned. This lifts the pre-standard cap.
2. Make the derivation hybrid — mix the classical ECDH result and the ML-KEM shared secret into the KDF. This lifts the non-hybrid cap and, because it binds the identity-authenticated material into the session, it simultaneously resolves CORE-CFG24. Two findings, one change.
3. Delete the session-secret print at p2p/rlpx/rlpx.go:533 and purge retained logs from any deployment that ran this code. This lifts the disclosure cap.
4. Correct the published and due-diligence representations to match the implementation. This lifts the representation cap, and costs nothing but candour.

Raise CIRCL to v1.6.3 or later while doing so (CORE-CFG01), and add tests for the handshake and KEM paths, which are currently at 0% coverage (CORE-CFG34).

Reaching PQR-4 (Consensus-enforced) is a materially larger step and a genuine fork: TxDataPQC would need a persisted ML-DSA signature field, verified in the state transition for accounts that opt into post-quantum protection, with the size, gas-cost and test work that implies. It is the right destination for a chain marketed on post-quantum security, and it is the only route by which the claim currently being made to the exchange becomes true.

# 5. Post-Quantum Posture Matrix

Per-primitive reality as implemented in source — the central subject of BloFin's request. "QR today?" = does this component provide quantum resistance for on-chain / consensus-authenticated state at present.

| Component | Scheme (as coded) | Library | Layer | QR? | Note |
|---|---|---|---|---|---|
| RLPx transport KEM | CRYSTALS-Kyber768 (NIST round-3) | CIRCL v1.6.1 kem/kyber/kyber768 | P2P handshake: mandatory, no negotiation | Partial | Genuinely wired in and unavoidable: session AES/MAC keys derive solely from the Kyber shared secret. But it is pre-standard Kyber, not FIPS-203 ML-KEM, and non-hybrid — the classical ECDH result is computed yet excluded from key derivation, so a flaw in the Kyber implementation has no classical fallback. Protects transport confidentiality only; provides no protection to on-chain state. |
| Transaction signatures (all types) | secp256k1 ECDSA | go-ethereum stock crypto | Consensus: enforced | No | Every transaction on the chain, including type 0x75, is authenticated by classical ECDSA via crypto.Ecrecover. This is the actual trust root of the chain and it is not quantum-resistant. |
| Dilithium signing package | CRYSTALS-Dilithium mode2 (round-3, NIST level 2) | CIRCL v1.6.1 sign/dilithium/mode2 | RPC endpoint only: not consensus | No | Verified at the RPC boundary then discarded; the Dilithium signature is never attached to the transaction and never reaches the pool, block validator or state transition. mode2 is the weakest of CIRCL's three parameter tiers. Package is absent entirely on the update-config branch. |
| Tx 0x75 signature hash | SHAKE256-256 | golang.org/x/crypto/sha3 | Consensus: but scoped to one tx type | N/A | Changes only which digest is ECDSA-signed for type 0x75. A different hash does not confer post-quantum signature security; the signature over it remains secp256k1. Transaction.Hash() (the txID) stays Keccak-256. |
| Address derivation | Keccak-256 (standard) / SHAKE256 (tx 0x75 only) | go-ethereum crypto / x/crypto/sha3 | Consensus: conditional branch | N/A | PubkeyToAddress, CreateAddress and CREATE2 are unmodified Keccak-256. SHAKE256 derivation applies only where tt == PQCTxType. The claim that all account addresses use SHAKE256 is not accurate. Creates the dual-address hazard in CORE-CFG15. |
| Block, state and receipt hashing | Keccak-256 | go-ethereum stock | Consensus: enforced | N/A | Entirely unmodified. No SHAKE256 or other post-quantum-adjacent hashing exists in block headers, the state trie, receipts or the txpool. |
| Validator / consensus-layer keys | BLS12-381 | Lighthouse (upstream) | Consensus layer | No | Standard Ethereum BLS validator keys, untouched by this fork. Not quantum-resistant, and outside the scope of the fork's PQC work. |

Read as a whole, the matrix shows a post-quantum programme that has been completed at the transport layer and only started at the signature layer. Kyber768 is mandatory, unavoidable and genuinely protects peer-to-peer session confidentiality. Everything that determines the state of the chain — which account is debited, whether a

transaction is valid, whether a block is accepted — is still authenticated by secp256k1 ECDSA. Substituting SHAKE256 for Keccak-256 in one transaction type's signature hash does not change that: the digest changed, the signature scheme did not. Consequently, an adversary with a cryptographically-relevant quantum computer would be able to forge transactions on this chain exactly as they could on unmodified Ethereum. The correct characterisation for due-diligence purposes is that Krown Network has quantum-resistant P2P transport, using a pre-standard parameter set, and classical on-chain security. Any representation that transactions are protected by Dilithium signatures is not supported by the code reviewed.

# 6. Live Network Verification

Krown Network is a live production chain, not a pre-launch codebase. Mainnet launched 3 January 2026 and, at the time of this check, had produced roughly 1.47 million blocks. The following was read directly from the client's declared public endpoint, https://mainnet.krown.network, on 1 August 2026 using read-only JSON-RPC calls. This closes several items that a source-only review would have had to leave pending, and it materially strengthens the standing of every source-level finding in this report.

| Check | Observed on mainnet | Result |
|---|---|---|
| eth_chainId | 0x7bf (1983) | CONFIRMED: matches both client documents. Note this is NOT the 3151908 configured throughout the audited repository; see CORE-CFG12. |
| net_version | 1983 | Consistent. |
| web3_clientVersion | Geth/v1.16.2-stable-dd105cbf/linux-amd64/go1.24.11 | CRITICAL CONFIRMATION: the production build identifies commit dd105cbf, the exact commit reviewed in this assessment. |
| eth_syncing | false | Node synced and serving. |
| eth_blockNumber | 0x166a42 (1,469,506) | Chain live and progressing. |
| Block tag: finalized | block 0x1669f5 | SUPPORTED: the finalized tag resolves, so BloFin's recommended deposit-crediting model is sound. |
| Block tag: safe | block 0x166a15 | SUPPORTED. |
| Gas limit (live block) | 0x3938700 (60,000,000) | CONFIRMED: matches the repository value and the genesis figure stated to BloFin. |
| admin_* namespace | method does not exist / not available | GOOD: admin namespace is not exposed publicly. |
| debug_* namespace | method does not exist / not available | GOOD for exposure. Note archive/trace is therefore NOT available on the public endpoint today; BloFin was told a dedicated trace endpoint will be provisioned at onboarding. |
| personal_* namespace | method does not exist / not available | GOOD: no account-management surface exposed. |
| miner_* namespace | method does not exist / not available | GOOD. |
| txpool_status | { pending: 0x0, queued: 0x0 } | Exposed but benign. Empty mempool at the time of sampling. |
| Beacon /eth/v1/node/version | Prysm/v7.1.3 (linux amd64) | DIRECT VERIFICATION (2026-08-03) — closes CORE-CFG07. Not attestation. |
| Consensus params (/config/spec) | slashing / rewards / cycle = mainnet preset | CLEAN — no weakening. ETH1_FOLLOW_DISTANCE 10 is the one exception, see CORE-CFG08. |
| Genesis allocation | 262 entries; 1 funded (full supply); 256 precompile-seed | No hidden mint. Single-key supply + allowlist concentration — CORE-CFG28. |

| Check | Observed on mainnet | Result |
|---|---|---|
| P2P port 3303 exposure | reachable from internet; no host firewall | *** Removes the bound assumed for CORE-CFG24 — the finding is presently reachable. |
| Genesis hash (block 0) | 0xd7b1…2f74 | CFG NINJA-VERIFIED (2026-08-03) — exact match to the value given to BloFin. Independent confirmation, not attestation. |
| Live build vs remediation branch | Geth/…-dd105cbf (live) vs remediation/cfg-round1 (fixed) | Round-1 fixes are committed and CFG Ninja-verified in source, but NOT yet deployed — the live fleet still runs dd105cbf. Deployment is the remaining near-term gate. |

**Reviewed code confirmed in production**

The production client reports build dd105cbf, which is the exact commit of the core-block branch reviewed throughout this report. Every source-level finding here therefore describes code that is running on Krown mainnet today — not a development branch, not a candidate build. This removes the usual audit caveat about whether the reviewed tree matches production.

Two conclusions follow. First, the public RPC surface is properly hardened: the admin, debug, personal and miner namespaces are all disabled, which is the correct posture and is recorded as a positive result in CORE-CFG23. Second, and less comfortably, the deployment configuration in the audited repository does not describe this network. Every manifest in the repository configures chain ID 3151908 (or 89127398 on one branch); mainnet runs 1983. The repository contains devnet and Kurtosis harness configuration, not the production manifest set — which means the configuration-derived findings in this report describe artefacts that are not what operates the live chain. That is itself the finding, and it is developed in CORE-CFG12 and CORE-CFG26.

## 7. Consensus Parameters vs Upstream Presets

Deviation of consensus/reward/slashing parameters from stock Ethereum mainnet presets. Weakening of slashing or reward parameters would be a finding; matching presets is the safe result.

| Parameter | Krown | Upstream preset | Assessment |
|---|---|---|---|
| Chain ID | 3151908 | project-specific | Stock Kurtosis/ethpandaops devnet default, apparently never customised. Briefing states 1983, which appears nowhere in the repository. Reconcile before integration — see CORE-CFG12. |
| Genesis gas limit | 60,000,000 | ~30–36,000,000 | Approximately 2x mainnet. Not a weakening of security parameters, but raises per-block execution and state-growth load; validate against actual node hardware and archive-node cost projections. |
| PRESET_BASE | mainnet | mainnet | Match. All beacon preset values inherit from the mainnet preset. |
| Validator balance | 32,000,000,000 Gwei | 32,000,000,000 Gwei | Match (32 ETH). |

| Parameter | Krown | Upstream preset | Assessment |
|---|---|---|---|
| Ejection balance | 16,000,000,000 Gwei | 16,000,000,000 Gwei | Match (16 ETH). |
| Churn limit quotient | 65,536 | 65,536 | Match. |
| Slot duration | 12 s | 12 s | Match. |
| Min withdrawability delay | 256 epochs | 256 epochs | Match. |
| Shard committee period | 256 epochs | 256 epochs | Match. |
| Slashing / inactivity quotients | not overridden | mainnet preset | Match. A repository-wide search for SlashingPenalty, InactivityPenalty, EffectiveBalance and SlotsPerEpoch returned no override in any config or Go source. No weakening — a clean result. |
| Eth1 follow distance | 2048 (trunk) / 10 (update-config) | 2048 | WEAKENED on the update-config branch only. Reducing follow distance from 2048 blocks to 10 sharply cuts the safety margin against execution-layer reorgs when the beacon chain reads deposit-contract state. Not present on the trunk — see CORE-CFG08. |
| Fork schedule (EL) | stock mainnet table | stock mainnet table | Unmodified through Prague. No Krown-specific chain-config object exists in params/config.go — consistent with the chain running on a generic devnet configuration rather than a purpose-built chainspec. |

# 8. Detailed Findings

Each finding uses the CFG Ninja Chain-Core catalog (CORE-CFG##). Status: Detected / Attention Required / Resolved / Pending Verification. Source-level items are cited by file:line on the reviewed commit.

## CORE-CFG06 | Kyber session secret written to standard output on every RLPx handshake.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Debug code in critical paths | Medium | 5.1 | p2p/rlpx/rlpx.go:533 (secrets(), reached from both runInitiator and runRecipient at rlpx.go:444 and rlpx.go:583); second instance at core/types/transaction.go:884 | Resolved (fix verified in source; deployment pending) |

BVSS 4.0 · Evidence grade E3

### Description

The function that derives RLPx session secrets prints the raw Kyber shared secret to standard output immediately before zeroing it. Because secrets() is invoked from both the initiator and recipient paths, this occurs on every peer connection the node makes or accepts. The value printed is the input from which the AES and MAC session keys are derived (rlpx.go:507-509), so anyone able to read the node's stdout — a log file, a container log stream, a log-aggregation platform, a support bundle, or a screen-shared terminal — can reconstruct the session keys for that connection and decrypt the peer traffic. The zeroBytes() call on the following line protects the value in memory but is defeated entirely by having already emitted it. A second, less severe instance of the same class exists at core/types/transaction.go:884, where an unconditional fmt.Println("PQC sigHash testing.....") sits inside TxDataPQC.sigHash(): a signing hot path. That one leaks no secret but writes to stdout on every PQC signature-hash computation and signals that the post-quantum path was never cleaned up for production. Present-day impact is bounded by the permissioned, single-operator fleet: today the peers are all Krown's own, so the confidentiality lost is Krown's own inter-node traffic. That bound disappears at the roadmap milestone admitting external validators, at which point this becomes a straightforward path to decrypting third-party peer sessions and should be treated as Critical.

### Recommendation

Delete the fmt.Println at rlpx.go:533 and the one at transaction.go:884. Neither should exist in a release build. Establish a policy that key material, shared secrets and seeds may never be passed to fmt.Print*, log.Print* or any logger at any level, and enforce it with a CI lint rule or a pre-commit grep over crypto/, p2p/ and core/types/. Rotate nothing — Kyber secrets are ephemeral per-session — but audit any retained node logs from deployments running this code and purge them, since historical session secrets remain recoverable from them. Add a build-time check that fails on fmt.Print* anywhere under p2p/ and crypto/.

### Mitigation / Status

FIX VERIFIED IN SOURCE 2026-08-03, live remediation pending rollout. On branch remediation/cfg-round1 (commit 65b7f88) both debug prints are removed — the fmt.Println of the derived Kyber session secret in handshakeState.secrets (rlpx.go) and the unconditional print in TxDataPQC.sigHash (transaction.go) — exactly two lines, one per file, imports unchanged. CFG Ninja confirmed both are present at dd105cb and absent on the branch. IMPORTANT: the live production fleet still runs commit dd105cbf (confirmed via web3_clientVersion), so production nodes CONTINUE to write the session secret to local logs on every handshake until the fixed image is built and rolled out. The exposure remains bounded as before — local json-file driver, 500 MB rotation ceiling, no aggregation agent — and the client has scheduled the log purge to coincide with the rollout, since the emitting code runs until then. This finding is therefore fixed in code and CFG Ninja-verified, but NOT resolved on the live network; it resolves on deployment, which CFG Ninja will re-verify.

Previous: Reasoning corrected 2026-08-03. One of the three grounds accepted on 2 August for reducing this from

High was that "every peer is Krown's own, so no third-party session traffic exists to be decrypted." The client's Q9 disclosure makes that premise unsound: the p2p port is open to the internet (see CORE-CFG24), so external peers can connect and their session secrets are written to node logs on every handshake. Held at Medium nonetheless, because exploitation still requires access to host logs, and the client has confirmed the json-file log driver rotates at a 500 MB ceiling (max-size 100m x max-file 5) with current files at 18-40 MB — a bounded retention window rather than indefinite. The retention bound is the reason this stays Medium and does not return to High.

Previous: Severity revised 2026-08-02 following client response. Present-day impact reduced on client evidence: inter-node traffic is block/attestation/tx gossip published within seconds, all peers are Krown-operated, and the production log driver is local json-file with no aggregation agent (verified by the client across four hosts). Held at Medium — the secret still permits decryption of any recorded session by a party with host or log access, and the finding escalates to Critical at the external-validator milestone.

Previous: Open. No mitigating control identified in the reviewed source. Not gated by a debug flag, a build tag, or a verbosity level.

**References:** CWE-532: Insertion of Sensitive Information into Log File | CWE-215: Insertion of Sensitive Information Into Debugging Code | OWASP ASVS V7.1.1

Evidence (reviewed source)

```
p2p/rlpx/rlpx.go:533
  fmt.Println("Shared secret: ", h.kyberSharedSecret)
  // Cleanup sensitive data
  zeroBytes(h.kyberSharedSecret)
  h.kyberSharedSecret = nil

p2p/rlpx/rlpx.go:507-509  (what the printed value protects)
  sharedSecret := crypto.Keccak256(h.kyberSharedSecret, crypto.Keccak256(h.respNonce, h.initNonce))
  aesSecret    := crypto.Keccak256(h.kyberSharedSecret, sharedSecret)
  s := Secrets{ AES: aesSecret, MAC: crypto.Keccak256(h.kyberSharedSecret, aesSecret), ... }

core/types/transaction.go:884  (second instance, no secret leaked)
  fmt.Println("PQC sigHash testing.....")
```

# CORE-CFG24 | RLPx handshake discards the peer identity-signature result — proof-of-possession absent.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| P2P peer permissioning & DoS | Medium | 5.3 | p2p/rlpx/rlpx.go:464-467 (handleAuthMsg — ecrecover result assigned to _), related paths at rlpx.go:608 | Acknowledged (scope-limited) |

BVSS 4.0 · Evidence grade E3

### Description

In the rewritten handshake the recipient recovers the initiator's public key from the supplied signature and then discards it, assigning the result to the blank identifier. In stock RLPx this recovery is load-bearing in two ways: it proves the initiator possesses the private key matching the claimed static identity, and it yields the ephemeral ECDH public key that feeds session-key derivation — so an impersonated identity naturally fails to derive matching keys and the connection dies. In this fork, session keys derive entirely from the Kyber public keys, which are transmitted in the clear and are never bound to the ECDSA identity signature. crypto.Ecrecover succeeds for essentially any well-formed 65-byte signature, so the remaining call is a syntactic check with no authentication value. The practical consequence is that a peer can present an arbitrary claimed identity in InitiatorPubkey and complete a handshake, because nothing verifies that it controls the corresponding key. That enables peer-identity spoofing and materially eases eclipse and man-in-the-middle positioning against the peer-selection and reputation logic that assumes node IDs are self-authenticating. This finding is reported as a strong source-level conclusion, not a demonstrated exploit: the code was not compiled or run, and the handshake was not exercised against a live node. Verification is listed in the pending items. SCOPE AND SEVERITY, restated 2026-08-03. This finding has two separable parts, and only one of them is within the agreed scope of this engagement.

IN SCOPE — the source defect. The handshake recovers the initiator's public key and assigns it to the blank identifier (rlpx.go:464), so possession of the claimed identity key is never verified. This is a source-confirmed authentication weakness (evidence grade E3) and it falls squarely within the reviewed scope of "RLPx handshake construction, authentication". CFG Ninja rates the reviewable defect at Medium: an absent peer proof-of-possession is a real design weakness in the transport authentication.

OUT OF SCOPE — the exploitability. How far the defect can be pushed in practice — peer-table pollution, eclipse positioning, defeat of peer banning — depends on network exposure (whether the p2p port is reachable, host firewalling, network segmentation) and can only be established by active network testing. That is penetration / infrastructure testing, not chain-core source review, and it was not commissioned at intake; this report's methodology records that no dynamic testing or proof-of-concept exploitation was performed. CFG Ninja therefore does NOT rate this finding on network exploitability it was not engaged to assess.

For context, not as a CFG Ninja-verified rating: the client's Q9 answer states that the p2p port is reachable from the internet with no host firewall and that they claim no compensating control at that layer. If that holds, a network/infrastructure assessment would likely rate the exploitable form of this finding materially higher, up to High, because an absent handshake authentication control on an internet-reachable port enables eclipse and MITM positioning. That assessment is recommended, and is the proper venue to determine the exploitable severity. It is out of scope here.

The client's root-cause account is accepted and consistent with the source: when the handshake was rewritten for mandatory Kyber encapsulation, session-key derivation moved onto the Kyber shared secret alone and the recovered identity key lost its consumer. Not deliberate. The fix is specified and sequenced ahead of Milestone 3.

### Recommendation

Restore binding between the recovered identity and the session. Concretely: capture the recovered public key, compare it against the claimed InitiatorPubkey, and abort the handshake on mismatch; and mix the identity-authenticated material into key derivation so that a forged identity cannot produce a working session — for example by including the

static-shared-secret / ECDH result alongside the Kyber shared secret in the KDF input, which also resolves the non-hybrid concern raised in CORE-CFG16. Then add a protocol-level regression test that attempts a handshake with a signature not corresponding to the claimed identity and asserts failure. Treat this as a blocker for the external-validator milestone. Because the exploitable severity of this defect turns on network exposure that is outside this engagement's scope, a separate network / infrastructure assessment is recommended to establish it — and if the p2p reachability the client describes is confirmed, to treat the remediation as urgent rather than milestone-gated.

**Mitigation / Status**

SCOPE-LIMITED REPORTING ELECTED, 2026-08-03 — path (b) of the two set out on 3 August. No active handshake test is authorised or performed. This finding is carried as an IN-SCOPE, source-confirmed authentication defect at Medium: the handshake does not verify proof-of-possession of the claimed identity (rlpx.go:464), which is certain from the code. Its EXPLOITABILITY — reachability, eclipse and MITM positioning — depends on network exposure and can only be established by active network testing, which is penetration/infrastructure work outside this engagement's agreed static-review scope; that work is not performed and its result is not asserted. A separate network / infrastructure assessment is recommended and is the proper venue to determine the exploitable severity, which on the client's own p2p-exposure disclosure (Q9) could be materially higher.

Because the finding is rated on the source-reviewable defect and not on an unproven exploit, the house rule against carrying a High at evidence grade E3/E4 into a final report is not engaged: this is a Medium at E3 (source-confirmed), reported within scope, not a High held on inference. No severity was reduced to reach an outcome — the Medium reflects what the source review substantiates, and the higher exploitable rating is expressly deferred to a differently-scoped assessment rather than withdrawn.

Client remediation is specified and sequenced ahead of Milestone 3, testnet-verified before the fleet; root-cause account accepted (the recovered key lost its consumer when derivation moved onto the Kyber shared secret). CFG Ninja will re-verify the fix at source when it lands. If the client later commissions the network test (path a), the finding can be raised to E1 on independent evidence.

**References:** CWE-345: Insufficient Verification of Data Authenticity | CWE-287: Improper Authentication | devp2p RLPx transport specification

Evidence (reviewed source)

```
p2p/rlpx/rlpx.go:464-467  (recovered identity discarded)
  _, err = crypto.Ecrecover(signedMsg, msg.Signature[:])
  ...

Session keys are derived only from Kyber material, never from the identity:
 rlpx.go:507-509  aesSecret := crypto.Keccak256(h.kyberSharedSecret, sharedSecret)
 rlpx.go:503-505  secrets() hard-fails only if h.kyberSharedSecret == nil

InitiatorPubkey / RandomKyberPubkey are transmitted in the clear as fixed-size fields:
 rlpx.go:389  InitiatorKyberPubkey [kyber.KyberPublicKeySize]byte
 rlpx.go:400  RandomKyberPubkey
 rlpx.go:403  CipherText

Source defect (in scope, E3): rlpx.go:464 recovered pubkey -> _ (discarded).
Exploitability (out of scope): client Q9, 2026-08-03 — "no host firewall on eight hosts
across four providers (ufw inactive, iptables default ACCEPT), and the execution-layer
p2p port is reachable from outside the mesh. We are not claiming a compensating control."
Recorded as client-disclosed context; not CFG Ninja-verified (network assessment not in scope).
```

# CORE-CFG14 | Post-quantum signature layer is verified then discarded — not enforceable at consensus.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| PQC enforcement location & efficacy | Medium | 5.4 | internal/ethapi/api.go:1536-1596 (SendRawTransactionWithDilithium); terminal call to SubmitTransaction at api.go:~1441 | Acknowledged |

BVSS 3.6 · Evidence grade E3

### Description

The SendRawTransactionWithDilithium RPC endpoint performs two genuine checks: an ECDSA message signature binding the sender address to a Dilithium public key, and a Dilithium signature over the transaction hash. Having verified both, it discards them and forwards the ordinary ECDSA-signed transaction to SubmitTransaction. The source comment states this plainly. The Dilithium signature and public key are never attached to the transaction object, never enter the transaction pool, and never reach block validation or the state transition. The consequence is that the post-quantum signature is an off-chain attestation with no consensus weight, and it is fully optional: the identical transaction bytes can be submitted through the standard eth_sendRawTransaction endpoint with no Dilithium material whatsoever and will be accepted identically. There is no account, transaction type, or code path that requires a post-quantum co-signature. This was confirmed negatively as well as positively — PQCTxType and 0x75 appear in only two files in the entire repository (core/types/transaction.go and core/types/transaction_signing.go), and searches for PQC, Dilithium and Shake across core/state_processor.go, core/state_transition.go, core/block_validator.go and core/txpool/validation.go return no matches at all. An adversary capable of breaking secp256k1 would be unaffected by the presence of this endpoint. This finding is the direct basis for CORE-CFG18: it is the technical fact that contradicts the due-diligence representation that signature verification uses Dilithium.

The client's pre-audit briefing anticipates this finding and offers a considered answer, which deserves to be engaged rather than restated as a defect. The briefing states that post-quantum enforcement "is architected to occur at the wallet layer, where the Dilithium signature is mandatory and gates value movement, rather than at the node/consensus layer", and that the node-side drop is therefore "expected behaviour given the wallet-layer enforcement model". The briefing is commendably transparent: it documents the drop explicitly, reproduces the source comment, and shows that TxDataPQC carries no Dilithium field. Nothing was concealed. The architecture is nevertheless not a security control at the chain level, for a single reason. A wallet-layer check binds only those who choose to use the wallet. The chain accepts any transaction carrying a valid secp256k1 signature, submitted through the standard eth_sendRawTransaction endpoint, with no Dilithium material of any kind — and it cannot distinguish such a transaction from one that passed the wallet's check, because nothing about the Dilithium verification is recorded in the transaction, the receipt, or the state. An adversary capable of forging ECDSA signatures is, by definition, the threat that post-quantum signatures exist to counter, and that adversary will not be using Krown's wallet. Wallet-layer enforcement protects the honest user's workflow; it does not protect the chain's assets. The briefing's related point; that secp256k1 is foundational to go-ethereum and cannot simply be removed without breaking EVM and wallet compatibility; is entirely correct, and this report does not suggest removing it. The standard answer to exactly this constraint is a hybrid model: an additional, consensus-verified post-quantum signature field on an opt-in account class, so that classical compatibility is retained while post-quantum authentication becomes a base-layer guarantee. The briefing lists that as roadmap. This finding is the gap between that roadmap and what the due-diligence response tells the exchange is already true today.

### Recommendation

Decide between two honest positions and implement one. Either (a) make the post-quantum layer real: extend TxDataPQC with a Dilithium signature field, persist it in the transaction encoding, and enforce its verification in the state transition or block validator for accounts that have opted into post-quantum protection — noting this is a consensus change requiring a fork, a size and gas-cost analysis, and full test coverage; or (b) retain the endpoint as an explicitly-labelled off-chain attestation service and correct every public and due-diligence representation to match, stating that on-chain transaction authentication is secp256k1 ECDSA. Option (b) is legitimate and defensible; what is not defensible is the current gap between the two. Until one is chosen, the endpoint should not be described as providing quantum-

resistant transaction security. Given the wallet-layer architecture the client describes, the most direct interim step is honesty about its boundary: state in the due-diligence materials that post-quantum signature enforcement is a wallet-layer property that a direct RPC submission bypasses, and that consensus authentication is secp256k1. That single correction costs nothing, is verifiable, and would resolve the most serious representation gap in CORE-CFG18.

**Mitigation / Status**

Severity revised 2026-08-02 following client response. Reduced from High: the representation gap is scored at CORE-CFG18 and rating the same substance twice at High overstates it. The technical finding stands — an integrator must know that consensus authentication is secp256k1 and that a direct eth_sendRawTransaction bypasses the wallet-layer check.

Previous: Open. Independently confirmed by two reviewers working separately from source.

**References:** CWE-807: Reliance on Untrusted Inputs in a Security Decision | NIST IR 8547 (Transition to Post-Quantum Cryptography Standards) | CWE-1240: Use of a Cryptographic Primitive with a Risky Implementation

Evidence (reviewed source)

```
internal/ethapi/api.go:1534-1535 (source comment)
  // The Dilithium signature is verified and then dropped;
  // only the ECDSA-signed transaction is forwarded

internal/ethapi/api.go (terminal line of SendRawTransactionWithDilithium)
  // All verifications passed, submit the ECDSA-signed transaction
  return SubmitTransaction(ctx, api.b, tx)

core/types/transaction.go:761 — TxDataPQC.rawSignatureValues() returns plain V, R, S *big.Int.
 There is no Dilithium signature field on TxDataPQC.
core/types/transaction_signing.go:485 — recovery for every tx type, including 0x75,
 funnels through recoverPlain() -> crypto.Ecrecover(): secp256k1.
```

# CORE-CFG18 | The two client documents contradict each other, and the one sent to the exchange is the inaccurate one.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| PQC representation accuracy | Medium | 5.8 | Client pre-audit briefing and BloFin due-diligence responses, versus internal/ethapi/api.go:1536, core/types/transaction_signing.go:250, crypto/dilithium/dilithium.go:9, crypto/kyber/kyber.go:9-11, k8s/values.yaml:23 | Resolved |

BVSS N/A · Evidence grade E2

### Description

A listing decision is being made on written representations, and this assessment had the unusual advantage of reading two of them: the pre-audit briefing Krown prepared for the auditor, and the due-diligence responses Krown sent to BloFin. They do not agree, and the pattern of disagreement is consistent — where they diverge, the internal briefing is accurate and the external response overstates. On transaction signatures the divergence is direct. The BloFin response states that for the post-quantum transaction type "signature verification uses Dilithium rather than ECDSA/secp256k1". The pre-audit briefing states the opposite, correctly and in detail: the Dilithium signature "is verified at RPC ingress, after which the plain ECDSA transaction is submitted to the mempool", TxDataPQC "stores only conventional ECDSA signature values", and "transaction authentication at the consensus layer therefore currently rests on ECDSA". The code agrees with the briefing. The statement made to the exchange is not correct. On QRNG the same pattern appears: present-tense in the BloFin response, correctly listed as Planned in the briefing, and absent from the code entirely (CORE-CFG10). On standards, both documents overstate. The BloFin response calls the primitives "NIST-standardized" and names FIPS 204 and FIPS 203 specifically; the implementation uses CIRCL round-3 dilithium/mode2 and kyber/kyber768, and the repository contains no ML-DSA or ML-KEM code at all, although both ship in the same pinned CIRCL version. The response also describes CIRCL as "audited", which is fair, but the pinned v1.6.1 carries an open advisory (CORE-CFG01). On SHAKE256 the finding runs the other way, and fairness requires saying so. The BloFin response scopes it correctly — SHAKE256 for the PQC transaction type, with standard transactions continuing to use Keccak-256; and that matches the code exactly. It is the internal briefing that overstates here, asserting that "SHAKE256 replaces Keccak256 for hashing and address derivation" as "a consensus-level change". It is not chain-wide; it is confined to one transaction type (CORE-CFG15). On the consensus client the two documents simply contradict each other, Lighthouse against Prysm v7.1.3, and the briefing flags this for reconciliation itself (CORE-CFG07). On committed secrets the briefing enumerates six mnemonics where seven exist (CORE-CFG19). CFG Ninja attributes no intent to any of this. The consistent explanation is documentation written against an intended architecture and not revised as implementation diverged, compounded by real branch drift; the Dilithium package exists on core-block and is deleted on update-config. But intent is not the issue for a counterparty. A reader of the BloFin response alone would conclude the chain has post-quantum transaction signatures, NIST-standardised primitives and quantum entropy in production. It has none of those three. Correcting that is the client's responsibility and it should happen before the listing proceeds, not after.

### Recommendation

Reconcile the two documents into a single technical statement of record and correct the BloFin response specifically. The minimum corrections: state that post-quantum signature enforcement is a wallet-layer property and that consensus authentication is secp256k1 ECDSA; identify the primitives as pre-standard round-3 CRYSTALS-Kyber768 and CRYSTALS-Dilithium mode2 with a stated migration plan to FIPS-203 and FIPS-204; move QRNG to a labelled roadmap section; publish one authoritative consensus-client identity; and correct the mnemonic count to seven with the seventh derived. The commercial argument for doing this is stronger than the compliance one. Every one of these corrections is survivable — a chain with post-quantum transport, a wallet-layer PQ signature product and a credible migration roadmap is a genuinely differentiated position, and the CFG Ninja Post-Quantum Readiness Grade in section 4 gives Krown a concrete route from PQR-2 to PQR-3 with four bounded changes. What is not survivable is an exchange discovering the gap after listing. CFG Ninja will verify the corrected representations against source in the final report and will record the correction as a resolved finding, which is a materially better artefact to hold than an uncorrected one.

**Mitigation / Status**

RESOLVED 2026-08-03. The client supplied corrected due-diligence text covering every representation item in this finding: transaction-signature position (consensus authentication is secp256k1; post-quantum enforcement is a wallet-layer property that a direct eth_sendRawTransaction bypasses), primitives and standards (round-3 CRYSTALS-Kyber768 / Dilithium mode2, not the finalised FIPS-203/204 standards, with migration planned), QRNG (RANDAO in production; QRNG2 a roadmap item), post-quantum transport (mandatory Kyber768, transport-only), consensus client (corrected to Prysm v7.1.3), and the validator count. CFG Ninja reviewed the corrected wording against source and confirms it is accurate — the primitives, the enforcement layer, the client identity and the randomness source now match the implemented reality.

Closure rests on two things stated plainly: CFG Ninja's verification that the corrected wording is accurate against source, and the client's attestation that the corrections have been applied to the responses provided to BloFin. CFG Ninja cannot independently confirm the text as delivered to the exchange, and recommends that BloFin be given the corrected responses directly and, if desired, this report as the independent basis for them. The alerting representation raised separately in CORE-CFG27 falls under the same correction and should be included.

Previous: A further representation mismatch surfaced 2026-08-03 (see new finding CORE-CFG27): the BloFin response describes 24-hour alerting on finality, reorg depth and the latest-vs-finalized gap, while the client's Q24 answer confirms that chain-level alerting is "the layer currently being built out." Same class as the other CORE-CFG18 items and to be corrected in the same pass. The client states the due-diligence responses are now corrected across consensus client, signature position, standards claims, QRNG, validator count and key-custody/monitoring; CFG Ninja will verify the corrected text against source and record this finding resolved on receipt.

Previous: Client has accepted this finding in full and supplied corrected text for the exchange response covering transaction signatures, primitives, QRNG, post-quantum transport, consensus client and validator count. CFG Ninja has reviewed the corrected wording against source and finds it accurate. This finding closes once the corrected text is published to BloFin — it requires no code change and is the single highest-value remediation available.

The pattern now holds on four counts, not three: the consensus-client reversal in CORE-CFG07 is a further instance in which the internal briefing was accurate and the document sent to the exchange was not.

Previous: Open. Client has been notified of the discrepancies through this assessment. No corrected documentation has been submitted for review at the time of writing.

**References:** FIPS 203 (ML-KEM) | FIPS 204 (ML-DSA) | CWE-1053: Missing Documentation for Design

Evidence (reviewed source)

```
Claim (BloFin DD 2.1): "signature verification uses Dilithium rather than ECDSA/secp256k1"
 -> CONTRADICTED by code (api.go:1536 verify-then-drop; transaction_signing.go:485 Ecrecover)
 -> CONTRADICTED by Krown's own briefing 3.A ("authentication ... currently rests on ECDSA")

Claim (BloFin DD ref card + 2.4): QRNG entropy via Quantum eMotion  [present tense]
 -> zero code matches on any branch; briefing s.6 lists QRNG2 under PLANNED  (CORE-CFG10)

Claim (BloFin DD 2.1/2.4): "NIST-standardized" ML-DSA FIPS 204 / ML-KEM FIPS 203
 -> dilithium.go:9  circl/sign/dilithium/mode2   (round-3, not ML-DSA)
 -> kyber.go:9-11   circl/kem/kyber/kyber768     (round-3, not ML-KEM)

Claim (BloFin DD 2.1) SHAKE256 scoped to the PQC tx type  -> ACCURATE, matches code
Claim (briefing 3.A) SHAKE256 is a chain-wide consensus change -> OVERSTATED (CORE-CFG15)

Consensus client: BloFin says Lighthouse, briefing says Prysm v7.1.3  (CORE-CFG07)
Mnemonics: briefing enumerates 6, scan finds 7                        (CORE-CFG19)
```

# CORE-CFG13 | Dilithium is pre-standard round-3 mode2, not FIPS-204 ML-DSA.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| PQC signature scheme | Low | 3.2 | crypto/dilithium/dilithium.go:9; go.mod:96 (github.com/cloudflare/circl v1.6.1) | Acknowledged |

BVSS 2.2 · Evidence grade E3

### Description

The signing package imports CIRCL's sign/dilithium/mode2, which is the NIST round-3 CRYSTALS-Dilithium submission, not the finalised FIPS-204 ML-DSA standard. CIRCL v1.6.1 does ship ML-DSA under sign/mldsa, so the standardised implementation was available in the pinned dependency and was not used. A repository-wide search for mldsa, ml-dsa and related identifiers returns no matches anywhere in the tree, confirming no ML-DSA code exists. Two consequences follow. First, round-3 Dilithium and FIPS-204 ML-DSA are not interoperable — signatures and keys produced by this code will not verify against standards-conformant implementations, which will complicate any future integration with a custodian, wallet or exchange that implements the standard. Second, mode2 is the lowest of CIRCL's three parameter tiers, corresponding to NIST security category 2; mode3 or mode5 would be the conventional choice for a system marketed on its post-quantum properties. Neither point is a vulnerability in the exploitable sense — the parameters are not broken; but for a chain whose distinguishing claim is post-quantum security, shipping the pre-standard scheme at its weakest tier is a meaningful gap between the claim and the implementation, and is the kind of detail an exchange's security review will check.

### Recommendation

Migrate to CIRCL's sign/mldsa (FIPS-204 ML-DSA) and select at minimum ML-DSA-65, equivalent to the former mode3, for a system positioning itself on post-quantum security. Because the current Dilithium layer is not consensus-enforced (CORE-CFG14), this migration can be made now at low cost — there is no on-chain state or historical signature to remain compatible with. Making the change before rather than after the signature layer becomes enforceable avoids a future migration under far harder constraints. Pin the CIRCL dependency to an exact version and record the chosen parameter set in the technical documentation.

### Mitigation / Status

Severity revised 2026-08-02 following client response. Reduced from Medium: round-3 Dilithium mode2 has no known cryptanalytic break, so this is a parameter-tier and standards-alignment finding rather than an exploitable defect. The representation issue is scored at CORE-CFG18.

Previous: Open. Note that crypto/dilithium is present on core-block but deleted on the update-config branch; the client should confirm which state is intended for production.

**References:** FIPS 204: Module-Lattice-Based Digital Signature Standard | NIST IR 8547 | CWE-327: Use of a Broken or Risky Cryptographic Algorithm

Evidence (reviewed source)

```
crypto/dilithium/dilithium.go:9
  mode2 "github.com/cloudflare/circl/sign/dilithium/mode2"

go.mod:96
  github.com/cloudflare/circl v1.6.1

CIRCL v1.6.1 also provides sign/mldsa (FIPS-204 ML-DSA) — available but unused.
Repository-wide search for mldsa | ml-dsa: no matches on any branch.
```

# CORE-CFG16 | Kyber KEM is pre-standard round-3 and non-hybrid; wire format breaks upstream interoperability.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| PQC transport / KEM | Medium | 5.1 | crypto/kyber/kyber.go:9-11, 27, 52-58; p2p/rlpx/rlpx.go:286, 389, 400, 403, 481-492, 503-509, 587-640, 619, 656 | Acknowledged |

BVSS 4.4 · Evidence grade E3

### Description

Three distinct issues in one component. First, the KEM is CIRCL's kem/kyber/kyber768 — round-3 CRYSTALS-Kyber, not FIPS-203 ML-KEM. CIRCL v1.6.1 provides kem/mlkem/mlkem768 and an ML-KEM-768 scheme identifier; neither is used, and no ML-KEM code exists in the tree. Second, the construction is non-hybrid. The classical ECDH static shared secret is computed but is used only for the identity signature check and is excluded from key derivation entirely: AES and MAC keys derive solely from the Kyber shared secret, and secrets() hard-fails if that secret is nil, so no code path produces a working session without it. Standard practice for post-quantum migration — and the basis of the deployed X25519+ML-KEM hybrids in TLS; is to combine classical and post-quantum secrets so that a flaw in either leaves the other protecting the session. Here a flaw in the CIRCL Kyber768 implementation would compromise transport confidentiality outright, with no classical contribution to fall back on. Third, there is no capability negotiation: msg.Version is hardcoded to 4 on both sides and never checked against peer capability, and no conditional branch for a non-Kyber peer exists. That is positive from a downgrade perspective; there is no downgrade path to attack; but it also means the fixed-size authMsgV4/authRespV4 schema no longer matches stock devp2p, so a vanilla go-ethereum node cannot complete a handshake with a Krown node. This is a liveness and interoperability constraint that must be understood by anyone planning to run standard tooling, bridges or indexers against the network.

### Recommendation

Migrate to CIRCL's kem/mlkem/mlkem768 (FIPS-203 ML-KEM-768). Make the construction hybrid by mixing both the classical ECDH result and the ML-KEM shared secret into the KDF — for example KDF(ecdh_shared || mlkem_shared || nonces): which simultaneously restores the identity binding that CORE-CFG24 identifies as missing, making these two fixes complementary. Remove the hardcoded Kyber security-level assumption: currentKyberLevel is a package global while the wire-field sizes are hardcoded for Kyber768, so changing the level anywhere would silently corrupt the handshake; either derive the sizes from the level or make the level a compile-time constant. Finally, document the wire incompatibility with stock devp2p explicitly for integrators, and if interoperability with standard clients is ever required, introduce a negotiated capability rather than a hard fork of the wire format.

### Mitigation / Status

Severity held at Medium 2026-08-02. The client correctly notes that round-3 Kyber768 has no known cryptanalytic break, and standards alignment alone would not warrant Medium. This finding is held on its other two elements: the NON-HYBRID construction, in which the classical ECDH secret is computed and then excluded from key derivation so that a flaw in one implementation removes transport confidentiality with nothing behind it — a defence-in-depth weakness, and the reason hybrid constructions are the deployed norm in the TLS post-quantum migration; and the wire incompatibility with stock devp2p, which is an operational constraint integrators must plan for.

Previous: Partially mitigating: the absence of negotiation means there is no downgrade attack surface. The non-hybrid design and pre-standard parameter set are unmitigated.

**References:** FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard | RFC 9370 / IETF PQUIP hybrid key-exchange guidance | CWE-327: Use of a Broken or Risky Cryptographic Algorithm

Evidence (reviewed source)

```
crypto/kyber/kyber.go:9-11
   github.com/cloudflare/circl/kem/kyber/kyber768   (round-3 Kyber, not FIPS-203 ML-KEM)

p2p/rlpx/rlpx.go:286     kyber.SetKyberSecurityLevel(kyber.Kyber768)   // forced every handshake
crypto/kyber/kyber.go:27  currentKyberLevel is a package-global var
crypto/kyber/kyber.go:52-58  KyberPublicKeySize=1184, KyberCiphertextSize=1088 hardcoded for 768

Non-hybrid derivation — classical ECDH excluded:
 rlpx.go:507-509  keys derive from h.kyberSharedSecret only
 rlpx.go:503-505  secrets() hard-fails if h.kyberSharedSecret == nil

No negotiation:
 rlpx.go:619, 656  msg.Version hardcoded to 4, never checked against peer capability
```

## CORE-CFG15 | One private key resolves to two different addresses depending on transaction type.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| PQC hashing & address derivation | Low | 3.4 | core/types/ transaction_signing.go:456-468 (addrFromRecoveredPub), 207-209 and 248-250 (useShakeAddr := tt == PQCTxType); core/types/ transaction.go:871-891 | Acknowledged |

BVSS 2.8 · Evidence grade E3

### Description

Sender-address derivation branches on transaction type. For type 0x75 the address is the truncation of SHAKE256 over the recovered public key; for every other type it is Keccak-256, as in stock Ethereum. Because both paths recover the same secp256k1 public key from the same ECDSA signature, a single private key controls two distinct addresses — one used when it signs a PQC transaction, another used for everything else. This is consensus-relevant, since the derived address determines which account is debited. The hazard is that essentially every piece of Ethereum infrastructure assumes the invariant that one key maps to one address: wallets will display and manage only the Keccak address, exchanges crediting deposits by derived address will not see funds arriving at the SHAKE address, block explorers will present the two as unrelated accounts, and any contract implementing access control by address will grant permission on one and deny it on the other. Funds sent to the SHAKE-derived address are recoverable only by signing a type-0x75 transaction, which is itself currently not admissible through the transaction pool (CORE-CFG04): so in the code as it stands, value reaching that address may be effectively stranded. The design is not documented as a dual-address model; the briefing describes only a SHAKE256 signature hash, which understates the consequence. The interaction with CORE-CFG04 is what keeps this at Medium rather than higher: the second address is hard to reach today precisely because the transaction type is unreachable. Fixing CORE-CFG04 without addressing this would raise the severity.

### Recommendation

Remove the dual-derivation model. The address derivation for a given public key should not depend on the transaction type that key happens to be signing — use Keccak-256 uniformly, as upstream does, and keep SHAKE256 confined to the signature hash if it is retained at all. Changing the hash used for the signing digest is sufficient for domain separation between transaction types and does not require changing address derivation. If a distinct post-quantum account namespace is genuinely intended, it should be an explicit, documented account type with wallet and explorer support, not an implicit side-effect of a hashing branch. Before any change, scan the chain for value or nonce activity at SHAKE-derived addresses to confirm nothing is already stranded. Add a test asserting that PubkeyToAddress and the sender recovered from every transaction type agree for the same key.

### Mitigation / Status

Severity revised 2026-08-02 following client response. Reduced from Medium: the client scanned 2,876 recovered sender public keys and found zero SHAKE-derived counterparts carrying balance, nonce or code; nothing is stranded. CONDITION: this returns to Medium if CORE-CFG04 is resolved without it, since the hazard is latent only while the type is unreachable.

Previous: Open. Practical exposure is currently limited by CORE-CFG04 rendering type 0x75 unreachable via the standard mempool.

**References:** CWE-843: Access of Resource Using Incompatible Type | EIP-55 / EIP-155 address and signing conventions | CWE-694: Use of Multiple Resources with Duplicate Identifier

Evidence (reviewed source)

```
core/types/transaction_signing.go:456-468
  func addrFromRecoveredPub(pub []byte, useShake bool) common.Address {
    if useShake { sha3.ShakeSum256(out, pub[1:]) } else { h = crypto.Keccak256(pub[1:]) }
    ...
  }

core/types/transaction_signing.go:249
  useShakeAddr := (tt == PQCTxType)

Standard derivation is unmodified:
 crypto/crypto.go:285-288  PubkeyToAddress -> Keccak256(pubBytes[1:])[12:]
 crypto/crypto.go:111-114  CreateAddress   -> Keccak256
 crypto/crypto.go:118-120  CreateAddress2  -> Keccak256
```

# CORE-CFG04 | Transaction type 0x75 can never be admitted by the transaction pool (uint8 shift overflow).

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Mempool & tx validation | Low | 3.1 | core/txpool/validation.go:29 and :48; core/txpool/legacypool/legacypool.go:558-564; core/types/transaction.go:41 | Acknowledged |

BVSS 2.0 · Evidence grade E3

### Description

Transaction-pool admission tests the transaction type against a bitmap: opts.Accept & (1 << tx.Type()). The Accept field is declared as uint8, and the legacy pool populates it with the legacy, access-list, dynamic-fee and set-code types. PQCTxType is 0x75 — decimal 117 — and does not appear in that bitmap. More fundamentally, it cannot: shifting 1 by 117 positions in a uint8 yields zero, so the acceptance test can never pass for this type regardless of what is added to the bitmap. The result is that the disclosed post-quantum transaction type is unusable through the normal submission path; a transaction of type 0x75 sent via eth_sendRawTransaction is rejected with ErrTxTypeNotSupported. A search across core/, core/txpool/, legacypool and blobpool found PQCTxType referenced only in core/types/transaction.go and core/types/transaction_signing.go, and nowhere in pool validation, which corroborates that the type was registered as a decodable envelope but never plumbed through admission. The severity is bounded because the failure mode is closed rather than open: transactions are rejected, not wrongly accepted. It is rated Medium rather than Low because it means a feature presented as shipped does not function, and because the fix necessarily touches transaction-admission logic, where a careless widening of the accept type could turn a closed failure into an open one. Note the interaction with CORE-CFG15: fixing this finding without fixing that one would make the dual-address hazard reachable in practice.

### Recommendation

Decide first whether transaction type 0x75 is intended to be usable. If it is, the accept mechanism must be widened beyond a uint8 bitmap — a map[uint8]struct{} or an explicit set is the natural fix, since the type space is 8-bit but the bitmap approach only supports types 0-7. Then add the type to every relevant pool's accept set, and add an end-to-end test that submits a type-0x75 transaction through eth_sendRawTransaction and asserts pool admission, propagation and block inclusion — the absence of such a test is why this defect shipped. Resolve CORE-CFG15 in the same change so that enabling the type does not activate the dual-address hazard. If the type is not intended to be usable, remove it from the codebase rather than leaving a non-functional consensus-adjacent code path in place, and correct the documentation that presents it as available.

### Mitigation / Status

Severity revised 2026-08-02 following client response. Reduced from Medium: fails closed, and no type-0x75 transaction has been admitted at any point in chain history — corroborated by CFG Ninja across 2,000 blocks sampled from genesis onward, and by the client across 45,593 transactions in the recent 50,000 blocks. A non-functional feature, not a security defect.

Previous: Fails closed. No unsafe state results; the transaction is rejected. No mitigating control needed for safety, but the feature is non-functional.

**References:** CWE-190: Integer Overflow or Wraparound | CWE-1284: Improper Validation of Specified Quantity in Input | EIP-2718: Typed Transaction Envelope

Evidence (reviewed source)

```
core/txpool/validation.go:29
  Accept uint8

core/txpool/validation.go:48
  if opts.Accept&(1<<tx.Type()) == 0 {
    return fmt.Errorf("%w: tx type %v not supported by this pool", core.ErrTxTypeNotSupported, tx.Type())
  }
```

Evidence (reviewed source) (continued)

```
core/txpool/legacypool/legacypool.go:558-564
   Accept: 0 | 1<<types.LegacyTxType | 1<<types.AccessListTxType |
         1<<types.DynamicFeeTxType | 1<<types.SetCodeTxType,

core/types/transaction.go:41
   PQCTxType = 0x75     // 117 decimal; 1<<117 on a uint8 evaluates to 0
```

# CORE-CFG07 | Consensus client confirmed Prysm v7.1.3 by direct verification.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Consensus client identity & version | Medium | 4.0 | kurtosis/network_params.yaml:17-18, 33-34; k8s/values.yaml:16; k8s/templates/statefulset-lighthouse.yaml:41-72; k8s/templates/deployment-validator.yaml; docker-compose/docker-compose.yml:143-231; configurations/setup-docker.sh:108-159 and setup-validator.sh:305-375 (update-config) | Resolved |

BVSS 2.0 · Evidence grade E2

### Description

This finding was reversed on 2 August 2026 following client evidence, and the reversal is instructive.

The execution client remains confirmed live: web3_clientVersion returns Geth/v1.16.2-stable-dd105cbf/linux-amd64/go1.24.11, pinning the production build to the exact commit audited here.

On the consensus client, the client has attested that the mainnet fleet runs Prysm v7.1.3 as both beacon node and validator client, on four hosts across four providers (Linode, Hetzner, Contabo, DigitalOcean), from images gcr.io/prysmaticlabs/prysm/beacon-chain:v7.1.3 and gcr.io/prysmaticlabs/prysm/validator:v7.1.3. Production consensus containers are pinned to an explicit version tag; the floating :latest tags are confined to the devnet manifests.

The client makes an argument from this report's own CORE-CFG12 which we accept: if the repository manifests establish what the devnet runs rather than what the fleet runs, they cannot be used to establish Lighthouse either. Our earlier reading — that the repository supported the Lighthouse claim — applied devnet evidence to a production question, and we have withdrawn it.

One qualification stands. This is client attestation, not independent verification. CFG Ninja could not reach the beacon layer; a direct probe of the beacon REST endpoint returned HTTP 404, which is consistent with the client's explanation that Prysm does not expose its REST API by default, but consistency is not confirmation. The finding closes when beacon access is provided (request A2).

The consequence for CORE-CFG18 is to strengthen it. The due-diligence response sent to BloFin states Lighthouse (Sigma Prime) and cites a configuration field; the internal pre-audit briefing states Prysm v7.1.3 and flags the discrepancy for reconciliation. The internal document was again the accurate one and the external document again incorrect — the fourth instance of that pattern. Client identity determines which CVE set and which client-specific consensus bugs apply, so an exchange performing due diligence against Lighthouse would validate the wrong advisory list entirely.

### Recommendation

Publish the corrected consensus-client identity in the BloFin response and in all technical documentation. Provide CFG Ninja with read access to a production beacon endpoint, or the output of /eth/v1/node/version and /eth/v1/config/spec, so the attestation can be independently confirmed and the live consensus parameters read (closes this finding and the live half of CORE-CFG08). Maintain explicit tag-and-digest pinning on the production consensus containers, and apply the same discipline to the devnet manifests so the two cannot be confused again.

**Mitigation / Status**

RECONCILED 2026-08-03. The Lighthouse configuration found throughout the repository manifests — the original basis on which this finding flagged a contradiction — is now explained: the pre-migration deployment ran Lighthouse. The production fleet migrated to Prysm v7.1.3, but the repository (and a stale bootstrap entry, see CORE-CFG26) retained the earlier Lighthouse configuration. A live production beacon confirms the operating fleet is Prysm with no Lighthouse peer in the connected set. This unifies the original CFG07 contradiction with the CORE-CFG12 root cause: the repository was never updated after the migration. Client is confirming the deployment history internally and updating the repository. Finding remains Resolved on the direct /eth/v1/node/version verification.

Previous: RESOLVED 2026-08-03 by direct verification, not attestation. /eth/v1/node/version on the running production beacon node returns "Prysm/v7.1.3 (linux amd64)". The due-diligence response to BloFin stated Lighthouse and has been corrected by the client. Production consensus containers are pinned to an explicit v7.1.3 tag (docker-compose.yml:52). Recorded as the fourth instance of the pattern in CORE-CFG18 where the internal briefing was accurate and the exchange document was not.

Previous: Production containers are pinned to an explicit v7.1.3 tag — our earlier :latest criticism applies to the devnet manifests only. CFG Ninja verification pending beacon access.

**References:** CWE-1104: Use of Unmaintained Third Party Components | CWE-1357: Reliance on Insufficiently Trustworthy Component | Ethereum client-diversity guidance

Evidence (reviewed source)

```
Live mainnet execution client (CFG Ninja, 2026-08-01 and re-verified 2026-08-02):
  web3_clientVersion -> Geth/v1.16.2-stable-dd105cbf/linux-amd64/go1.24.11

Consensus client — CLIENT ATTESTATION, 2026-08-02 (not independently verified):
  bn  gcr.io/prysmaticlabs/prysm/beacon-chain:v7.1.3
  vc  gcr.io/prysmaticlabs/prysm/validator:v7.1.3
  el  krown-local:latest
  four hosts across Linode, Hetzner, Contabo, DigitalOcean

CFG Ninja beacon probe, 2026-08-02: HTTP 404 / unreachable — consistent with Prysm not
exposing the REST API by default. Consistent, but not confirmation.

Client documents disagree; the external one is wrong:
  BloFin DD response  : Lighthouse (Sigma Prime)     INCORRECT
  Pre-audit briefing D: Prysm v7.1.3, flagged CONFIRM  CORRECT

2026-08-03 — /eth/v1/node/version -> {"data":{"version":"Prysm/v7.1.3 (linux amd64)"}}
Production manifest: bn image gcr.io/prysmaticlabs/prysm/beacon-chain:v7.1.3 (explicit tag).
```

# CORE-CFG10 | QRNG quantum-entropy capability is presented to the exchange as shipped; no implementation exists in any branch.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Proposer/committee selection & randomness | Medium | 4.5 | Repository-wide across all 8 branches; BloFin due-diligence response, Network Reference Card ("Post-quantum layer") and section 2.4; pre-audit briefing section 6 ("Planned") | Resolved |

BVSS N/A · Evidence grade E2

### Description

The due-diligence response supplied to BloFin describes the network's post-quantum layer as "Lattice-based ML-DSA signatures (NIST FIPS 204), QRNG entropy via Quantum eMotion partnership", and states in section 2.4 that "entropy hardening is additionally provided through our partnership with Quantum eMotion (QRNG)". Both statements are written in the present tense, in a document whose purpose is to describe what the exchange would be listing. No implementation exists. An exhaustive case-insensitive search across all eight branches for QRNG, Quantum eMotion, qemotion, quantum entropy and randomness beacon returns zero matches — not a stub, not a configuration flag, not a comment. Proposer selection and on-chain randomness are inherited unmodified from upstream: RANDAO handling appears only in the stock beacon/engine and core/vm files, with no Krown modification. The client's own pre-audit briefing is accurate on this point and places QRNG correctly. Its section 6 lists "QRNG2 as a verifiable quantum-randomness beacon feeding the chain" under Planned, alongside full consensus-level Dilithium enforcement, and explicitly not under Current. The internal document and the external document therefore disagree, and once again the accurate one is the internal briefing while the one sent to the exchange overstates. That is the same pattern identified in CORE-CFG14 and CORE-CFG18, and the repetition is what elevates this from a wording slip to a disclosure concern. Nothing here suggests the partnership does not exist or that the roadmap item is not genuine. The finding is narrowly that a planned capability is described to a counterparty as a current one, in the same sentence as capabilities that are real, which makes it difficult for a reader to tell them apart. For randomness specifically the practical impact today is nil — inheriting Ethereum's RANDAO is a perfectly sound default and no weakness was found in it; so this is rated for the representation, not for a technical defect.

### Recommendation

Move the QRNG statement out of the present-tense description of the current post-quantum layer and into a clearly labelled roadmap section, matching the treatment already used correctly in the pre-audit briefing. If a Quantum eMotion integration is contracted or in development, say so with that framing and a target. When it is implemented, expect the audit to assess it against CORE-CFG10 specifically: entropy source integrity, how quantum randomness is verified on-chain rather than trusted, bias and grinding resistance, and the failure mode when the beacon is unavailable — a randomness beacon that a single provider can stall or bias is a consensus liveness and fairness risk, not merely a feature. Until then, the honest statement is that proposer selection uses standard Ethereum RANDAO, which is a defensible position requiring no apology.

### Mitigation / Status

RESOLVED 2026-08-03. The QRNG representation is corrected in the client's due-diligence text, which CFG Ninja reviewed against source and found accurate: proposer selection and on-chain randomness use standard Ethereum RANDAO in production, and QRNG2 (the Quantum eMotion beacon) is stated as a roadmap item rather than a shipped capability. This matches the code, where no QRNG or Quantum eMotion implementation exists on any branch and RANDAO is inherited unmodified. Closure rests on CFG Ninja's verification of the corrected wording plus the client's attestation that it has been applied to the BloFin responses (see CORE-CFG18); it is client-attested as delivered, not independently confirmed. If a QRNG beacon is later implemented it will be assessed against CORE-CFG10 on its own merits (entropy source integrity, on-chain verifiability, bias/grinding resistance, liveness).

Previous: No technical defect. Standard upstream RANDAO is unmodified and sound. The finding is the disclosure gap.

**References:** CWE-1053: Missing Documentation for Design | Ethereum RANDAO / proposer selection specification | NIST SP 800-90B (entropy source validation)

Evidence (reviewed source)

Search across all 8 branches, case-insensitive:
pattern: qrng | quantum.?emotion | qemotion | randomness.?beacon | quantum.?entropy
core-block, main, dev, dev-test, staging, dilit-multi-sig, docker-k8s-config, update-config
RESULT: no matches on any branch.

RANDAO references are stock upstream only:
beacon/engine/gen_blockparams.go, beacon/engine/types.go,
beacon/types/exec_payload.go, core/vm/evm.go, core/vm/jump_table.go

Client documents:
BloFin DD, Network Reference Card : "QRNG entropy via Quantum eMotion partnership" [present tense]
BloFin DD, section 2.4 : "Entropy hardening is additionally provided..." [present tense]
Pre-audit briefing, section 6 : "QRNG2 ... " listed under PLANNED [accurate]

# CORE-CFG12 | The repository contains no production chain configuration — every manifest describes a devnet.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Genesis & fork schedule | Medium | 4.3 | k8s/values.yaml:23 and :31; kurtosis/network_params.yaml:74; docker-compose/config/genesis/values.env.template:2; docker-compose/docker-compose.yml:89; configurations/config/values.env:2 and setup-docker.sh:90 (update-config) | Resolved |

BVSS 3.2 · Evidence grade E2

### Description

This finding was substantially revised after live verification, and the revision matters: an earlier reading of the repository alone suggested the briefed chain ID of 1983 was simply wrong. It is not. Mainnet genuinely runs chain ID 1983, confirmed by direct RPC query, and Krown is registered in the ethereum-lists/chains public registry under eip155:1983. Both client documents are correct on this point and there is no replay-exposure concern at the network level. The actual finding is different and, for audit purposes, more significant. Every deployment artefact in the reviewed repository configures chain ID 3151908 — the stock default of the public ethpandaops/ethereum-genesis-generator and Kurtosis ethereum-package devnet tooling — across k8s/values.yaml, kurtosis/network_params.yaml, the docker-compose genesis template and the docker-compose networkid flag. One branch, update-config, uses a third value, 89127398. The value 1983 does not appear anywhere in the repository on any branch. The repository therefore contains devnet and test-harness configuration, not the manifest set that operates the production chain. The execution-client SOURCE in this repository is unambiguously production code; the live node reports build dd105cbf, the exact reviewed commit; but the CONFIGURATION alongside it is not. That split has a direct consequence for the scope of this assessment and for anyone relying on it: every configuration-derived observation in this report, including the gas limit, the Eth1 follow distance, the consensus-client selection and the committed credentials in k8s/values.yaml, describes devnet artefacts whose relationship to the production fleet is unestablished. Some happen to match production; the 60,000,000 gas limit is confirmed live; but that is fortuitous rather than evidence that the manifests are the production ones. A further consequence: the params/config.go fork schedule is the unmodified upstream mainnet table with no Krown-specific chain-config object. For a chain running its own chain ID in production, the absence of a committed chainspec means the authoritative definition of the network's consensus configuration lives outside version control, wherever the production manifests actually are.

### Recommendation

Bring the production configuration into version control. Commit the manifest set that actually operates mainnet — with chain ID 1983 — and add a Krown-specific chain-config object to params/config.go rather than relying on the upstream mainnet table. Clearly segregate the devnet and Kurtosis harness configuration so it cannot be mistaken for deployable production config, which is the same remediation the client's own briefing recommends at item E. For the exchange integration specifically, publish the chain ID, genesis hash and fork schedule in a document an integrator can verify against a running node; the genesis hash was supplied to BloFin and should be reproducible from the committed configuration, which today it is not. Reconcile or retire the divergent 89127398 value on update-config. Until the production manifests are in the repository, no configuration finding in any audit of this repository; including this one; can be represented as describing the live network.

### Mitigation / Status

RESOLVED 2026-08-03, verified on branch remediation/cfg-round1 (commits ae009d9, 46b7dfe). CFG Ninja independently verified this on the remediation/cfg-round1 branch (4 commits off dd105cb): the branch was fetched from origin and the diffs and build checked directly, not accepted on the commit hashes. params/config.go now defines KrownChainConfig — ChainID 1983, terminal total difficulty 0, all forks active from genesis (post-merge launch), deposit contract 0x4242…4242 — registered in NetworkNames as "krown", additive (nothing selects it yet). deploy/production/ now holds the production manifest set, separated from the devnet configuration in docker-compose/, k8s/ and kurtosis/, with host-specific values as deploy-time placeholders (MESH_IP, P2P_HOST_IP, CHECKPOINT_SYNC_URL)

and secrets mounted, not committed. The production consensus definition is now in version control; the network is reproducible from the repository. The committed config carries ETH1_FOLLOW_DISTANCE 10, matching what production actually runs (see CORE-CFG08) — the client committed reality rather than intention, which is correct.

Previous: Reframed 2026-08-02 as an auditability finding rather than a hygiene failure. The client's reason for holding production configuration outside the repository is legitimate — it carries the JWT secret, keystore password and validator seed, and committing those would be a worse finding than this one. Held at Medium because the consequence is unchanged: no third party, CFG Ninja included, can verify what configuration operates the production chain, and the genesis hash supplied to BloFin is not reproducible from anything committed. The client's stated remediation — committing manifest structure with secrets injected at deploy time, plus a Krown-specific chain-config object in params/config.go — closes it.

Previous: The chain ID itself is correct and publicly registered; there is no defect in the deployed network. The finding concerns what is and is not auditable from the repository.

**References:** EIP-155: Simple Replay Attack Protection | ethereum-lists/chains registry (eip155:1983) | CWE-1188: Insecure Default Initialization of Resource

Evidence (reviewed source)

```
Live mainnet (https://mainnet.krown.network, 2026-08-01):
  eth_chainId  -> 0x7bf  (1983)     net_version -> 1983

Audited repository — every manifest, all branches:
  k8s/values.yaml:23,31                          3151908
  kurtosis/network_params.yaml:74                3151908
  docker-compose/config/genesis/values.env.template:2    3151908
  docker-compose/docker-compose.yml:89           --networkid=3151908
  configurations/config/values.env:2     (update-config) CHAIN_ID="89127398"

Exhaustive search for 1983 / 0x7bf across all 8 branches: no configuration occurrence.
3151908 = stock default of the public ethpandaops devnet generator.
```

# CORE-CFG19 | Seven committed mnemonics — enumeration gap closed; the seventh derived and cleared.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Committed secrets | Medium | 4.9 | docker-compose/config/genesis/values.env.template:4; kurtosis/static_files/assertoor-config/tests/validator-lifecycle-test.yaml:6; configurations/config/values.env:4; configurations/setup-validator.sh:215, 217, 219, 221 (update-config); plus jwtsecret, keymanager.txt and k8s/values.yaml:19 | Resolved |

BVSS 4.1 · Evidence grade E2

### Description

The client's pre-audit briefing addresses this area directly and well. It states that six distinct mnemonics appear in deployment and test configuration, that each was expanded via EIP-2333/2334 derivation, that the resulting BLS public keys were compared against all 77 active mainnet validators, and that none controls a live validator. The methodology described is sound, the derivation implementation was validated against the official EIP-2333 test vectors, and the conclusion is exactly the reassurance an exchange needs. This assessment does not dispute any of it. It does, however, identify a gap in its scope. An independent scan across all eight branch references and all 169 reachable commits found SEVEN distinct mnemonics, not six. The briefing's table lists: the canonical BIP-39 all-zero-entropy vector, the Hardhat/Anvil default, two repeated-word filler placeholders, a Krown local-devnet seed used by the Kurtosis/genesis harness, and a test/devnet seed used by the assertoor test fixtures. Those six correspond exactly to six of the seven found here. The seventh — held at configurations/config/values.env:4 and configurations/setup-docker.sh:68 on the update-config branch, identified in this report by fingerprint 74dfdb1bfca4 — does not appear in the briefing's enumeration and was therefore not among the seeds derived and checked against the validator set. That matters because of which one it is. Six of the seven are recognisable public test vectors or obvious placeholders that carry no secret value by construction. The seventh is the only one that is not attributable to any known public tool. It is precisely the item that a derivation check exists to clear, and it is the one the check did not cover. This is not a contradiction of the client's conclusion; it is very possible that this seed also controls nothing; but the assurance currently offered to BloFin rests on an enumeration that is incomplete, and it should not be relied upon until the seventh is derived. Non-mnemonic credentials are also committed: a shared JWT authentication secret matching the well-known public Kurtosis sample value, a Prysm keystore password that is a single common dictionary word, a PKCS12 passphrase used only to encrypt a self-signed TLS certificate, and repeated-digit placeholders in a beacon-chain config template. No PEM private-key blocks, raw secp256k1 keys or cloud API credentials were found anywhere. Actual secret values are deliberately excluded from this report; each item is identified by location and by a truncated SHA-256 fingerprint, which is sufficient for the client to match an item without this document carrying the credential. Note also that .gitignore is stock go-ethereum boilerplate on every branch and covers none of the devnet tooling directories, *.env files, jwtsecret or keymanager files; these credentials are not leaked past a control, they are intentionally tracked, as CONFIG.md:32 states. One further observation, offered directly to the client rather than as a code finding: the due-diligence response document sent to BloFin contains a plaintext password for a third-party protocol report hosted at scalexa.ai. Credentials should not travel inside due-diligence documents that are forwarded between organisations. That password is not reproduced here and should be rotated.

### Recommendation

Derive the seventh mnemonic (fingerprint 74dfdb1bfca4) using the same EIP-2333/2334 methodology already documented in the briefing, compare the resulting BLS public keys and EOA addresses against the live validator set and against Ethereum mainnet for any deposit or balance, and publish the completed seven-row table so the assurance is whole. Treat it as compromised until cleared and rotate it regardless, since it has sat in a repository with an unknown reader set. Then adopt the handling the client's own briefing already recommends and extend it: move all seeds to injected environment variables or sealed secrets, add values.yaml, *.env, jwtsecret and keymanager patterns

to .gitignore, generate a fresh JWT secret per deployment instead of shipping the public Kurtosis sample, replace the dictionary-word keystore password, and add secret scanning to CI configured to fail the build. Rotate the scalexa report password. Finally, revise CONFIG.md: documenting that secrets are committed for reproducibility is honest, but it should state unambiguously that no value in the repository may ever be used on a network with value.

### Mitigation / Status

RESOLVED 2026-08-02. The client derived the previously unenumerated seventh seed and supplied the full result. Method verified by CFG Ninja as sound: BIP-39 to seed, EIP-2333/2334 at m/12381/3600/i/0/0 for indices 0-255, secp256k1 EOAs at m/44'/60'/0'/0/i, with the derivation implementation validated against the official EIP-2333 test vectors before use. Result: 0 of 256 derived BLS public keys in the live validator set (77 validators), 0 of 20 derived EOAs with balance or nonce. The seed controls nothing on the network and is being rotated regardless, on the basis that it sat in a repository with an unknown reader set. The supplied key list makes the comparison independently reproducible. Recurrence prevention (.gitignore coverage, CI secret scanning) remains outstanding.

**References:** CWE-798: Use of Hard-coded Credentials | CWE-540: Inclusion of Sensitive Information in Source Code | EIP-2333 / EIP-2334: BLS12-381 key generation and path derivation

Evidence (reviewed source)

```
Identified by location and truncated SHA-256 fingerprint only. Secret values are deliberately
not reproduced in this report; they can be matched by fingerprint or supplied under separate cover.

 #1 00195360685a  docker-compose/config/genesis/values.env.template:4     public Kurtosis test vector
            (also docker-compose.yml:66, k8s/values.yaml:18, kurtosis constants.star:117)
 #2 878588cb1d53  kurtosis/.../validator-lifecycle-test.yaml:6          public assertoor test fixture
 #3 74dfdb1bfca4  configurations/config/values.env:4                    UNATTRIBUTED - priority item
            (also configurations/setup-docker.sh:68; update-config branch only)
 #4 c557eec878df  configurations/setup-validator.sh:215                 canonical public BIP-39 vector
 #5 f79cd64f9cea  configurations/setup-validator.sh:217                 canonical public dev vector
 #6 188b2d3461fe  configurations/setup-validator.sh:219                 repeated-word placeholder
 #7 b605d204504f  configurations/setup-validator.sh:221                 repeated-word placeholder

 JWT 9af751eab90e  docker-compose/static/jwt/jwtsecret:1, kurtosis/.../jwt/jwtsecret:1,
            k8s/values.yaml:20   -- matches the public Kurtosis sample JWT value
 P12 3966a1157c90  kurtosis/static_files/keymanager/keymanager.txt:1  -- self-signed TLS cert only
 Keystore password: single dictionary word -- k8s/values.yaml:19, docker-compose.yml:67,
            configurations/setup-docker.sh:66, setup-validator.sh:233

The client briefing's derivation table enumerates 6 seeds. By location and classification
they map to items #1, #2, #4, #5, #6 and #7 above. Item #3 (74dfdb1bfca4) does not appear
in that enumeration and was therefore not derived against the live validator set.

Scope: 8 branch refs, 169 commits, full history. Read-only; no writes to the client repository.

CLIENT DERIVATION, 2026-08-02 — reviewed and accepted:
 item #3, configurations/setup-docker.sh:68 + configurations/config/values.env:4 (update-config)
 24 words, valid BIP-39 checksum. Client fingerprint 6a555860eebb vs our 74dfdb1bfca4 —
 differs by normalisation; location and word count confirm the same item.
 0 / 256 derived BLS keys in the live set       0 / 20 EOAs with balance
 0 / 20  EOAs with nonce                        derivation validated vs EIP-2333 vectors
 Live validator set: 77 (report previously stated 75).
```

# CORE-CFG23 | Public RPC namespace exposure: verified correctly hardened.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| RPC exposure & method hardening | Informational | N/A | internal/ethapi/api.go:1536-1596 (SendRawTransactionWithDilithium); related local policy gate at api.go:1441-1444, cmd/utils/flags.go, eth/ethconfig/config.go | Resolved |

BVSS N/A · Evidence grade E2

### Description

Recorded as a positive, live-verified result. The public endpoint was probed for the namespaces that most commonly cause incidents on EVM nodes, and all of them are disabled: admin, debug, personal and miner each return "method does not exist / is not available". That is the correct production posture and it is not the default — it reflects a deliberate configuration decision. txpool_status is reachable and returns pending and queued counts, which is benign and commonly left enabled. Two qualifications, neither adverse. Because the debug namespace is disabled, debug_traceTransaction and debug_traceBlockByNumber are not available on the public endpoint today; the client has told BloFin a dedicated, IP-allowlisted archive and trace endpoint will be provisioned during integration onboarding, and that commitment is consistent with what was observed. And this check covers the public endpoint only — the internal and any BloFin-dedicated endpoints were not in scope and should be verified separately, since a dedicated endpoint with the debug namespace enabled is precisely the kind of surface that needs its own allowlisting and rate limiting. The one item that remains open in this area is the unauthenticated Dilithium RPC method, which performs two CIRCL Dilithium verifications before any economic cost is incurred by the caller. Lattice verification is markedly more expensive than ECDSA recovery, so on an endpoint without per-method rate limiting this is a cheap way to consume node CPU. Rate limiting could not be assessed from outside and is listed for the final report.

### Recommendation

Maintain the current namespace posture, and apply the same discipline to the dedicated BloFin endpoint when it is provisioned: enable the debug namespace only there, IP-allowlist it, and rate-limit it. Add per-method rate limiting for SendRawTransactionWithDilithium specifically, bound the accepted input size before invoking lattice verification, and order the cheapest check first. Re-verify namespace exposure on every endpoint after the integration endpoints are stood up, since this posture is a configuration property and configuration drifts.

### Mitigation / Status

Verified correct on the public endpoint. Two observations from the production manifest (2026-08-03), neither an exposure on current evidence: (1) --http.corsdomain=*, --http.vhosts=*, --ws.origins=* and --authrpc.vhosts=* combined with network_mode: host mean the sole protection on the internal RPC is that the mesh address is not routable — client to confirm mesh addresses are unreachable from untrusted networks, since vhosts=* also removes DNS-rebinding protection. (2) --http.api / --ws.api list "engine"; verified against source that eth/catalyst/api.go:39 registers the engine namespace Authenticated:true and node/rpcstack.go filters authenticated namespaces off the public servers, so it is NOT served on 8545/8546 — a configuration smell producing a startup error line, not an exposure. Recommend removing "engine" from both lists.

Previous: Verified correct on the public endpoint as at 1 August 2026. No exposed administrative surface found.

**References:** CWE-770: Allocation of Resources Without Limits or Throttling | CWE-400: Uncontrolled Resource Consumption | JSON-RPC endpoint hardening guidance

### Evidence (reviewed source)

Read-only JSON-RPC probe, https://mainnet.krown.network, 2026-08-01:
admin_nodeInfo -> method does not exist/is not available GOOD
admin_peers -> method does not exist/is not available GOOD
debug_traceBlockByNumber -> method does not exist/is not available GOOD (trace not public)
personal_listAccounts -> method does not exist/is not available GOOD
miner_setEtherbase -> method does not exist/is not available GOOD
txpool_status -> {"pending":"0x0","queued":"0x0"} benign

Open item: internal/ethapi/api.go:1536 SendRawTransactionWithDilithium performs two CIRCL Dilithium mode2 verifications before any economic cost; rate limiting not observable externally.

# CORE-CFG08 | Weakened Eth1 follow distance is in the PRODUCTION consensus config, not a retiring branch.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Consensus parameters vs presets | Medium | 4.2 | configurations/config/values.env:44 and configurations/setup-validator.sh (update-config branch); compare docker-compose/config/genesis/values.env.template:35 and k8s/values.yaml:64 on the trunk | Acknowledged |

BVSS 3.4 · Evidence grade E3

### Description

Corrected and upgraded 2026-08-03 on production evidence. An earlier revision of this report stated that the weakened ETH1_FOLLOW_DISTANCE of 10 was confined to the update-config branch and that the trunk carried the mainnet standard of 2048. That was wrong, and wrong in the client's favour: the value 10 is in the LIVE production consensus configuration, confirmed in config.yaml:84 and in the running beacon node's /eth/v1/config/spec (ETH1_FOLLOW_DISTANCE 10).

The clean part of the earlier finding stands and is reconfirmed from the full live spec: slashing, reward and validator-cycle parameters all match the mainnet preset with no weakening — PRESET_BASE mainnet, BASE_REWARD_FACTOR 64, PROPOSER_SCORE_BOOST 40, MIN_SLASHING_PENALTY_QUOTIENT_BELLATRIX 32 / ELECTRA 4096, PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX 3, INACTIVITY_PENALTY_QUOTIENT_BELLATRIX 16777216, CHURN_LIMIT_QUOTIENT 65536, MAX_EFFECTIVE_BALANCE and EJECTION_BALANCE at 32/16 ETH, SLOTS_PER_EPOCH 32, SECONDS_PER_SLOT 12.

The follow distance is the exception. Reducing it from 2048 to 10 means the beacon chain treats execution-layer deposit-contract state as settled after ten blocks. The client's mitigation — a single operator, no competing proposers, and no observed execution-layer reorg deeper than one block (Q25) — is reasonable for present conditions but is an absence of adversaries rather than a control, and it ceases to hold when external operators arrive. The client disclosed this themselves and has committed to raising it to the mainnet standard ahead of Milestone 3.

### Recommendation

Raise ETH1_FOLLOW_DISTANCE to the mainnet standard of 2048 in the production config before external operators can propose, and add a configuration test asserting the deployed consensus parameters match the intended preset so a weakened value cannot reach production unnoticed again. Retain the current practice of not overriding slashing and reward parameters — that part is exemplary.

### Mitigation / Status

Client decision recorded 2026-08-03: ETH1_FOLLOW_DISTANCE stays at 10 for now and is raised to the mainnet standard 2048 ahead of external operators. The reasoning is accepted as sound: it is a consensus parameter, so the change must move every beacon node together rather than by rolling restart — a coordinated fleet-wide operation with its own risk — and the exposure (a deep execution-layer reorg reverting deposit data the consensus layer has acted on) is not realistic under a single operator with no competing proposers and no observed reorg deeper than one block. CFG Ninja reclassifies this from a near-term to a STAGE-GATED condition, alongside CORE-CFG06 and CORE-CFG24, all remediated as part of the pre-external-operator work. The finding stands open until then, by the client's own acceptance.

Previous: Open. Bounded in present conditions by single-operator control with no observed deep reorgs, but that is not a control and does not survive the external-validator milestone. Client has committed to remediation ahead of Milestone 3.

**References:** Ethereum consensus specifications: ETH1_FOLLOW_DISTANCE | CWE-1188: Insecure Default Initialization of Resource

Evidence (reviewed source)

> Live production consensus config (2026-08-03):
> config.yaml:84 ETH1_FOLLOW_DISTANCE: 10
> /eth/v1/config/spec "ETH1_FOLLOW_DISTANCE":"10"
> vs mainnet standard 2048
>
> Clean, from the same live spec — no weakening of slashing/rewards/cycle:
> PRESET_BASE mainnet · BASE_REWARD_FACTOR 64 · PROPOSER_SCORE_BOOST 40
> MIN_SLASHING_PENALTY_QUOTIENT_BELLATRIX 32 / ELECTRA 4096
> PROPORTIONAL_SLASHING_MULTIPLIER_BELLATRIX 3 · CHURN_LIMIT_QUOTIENT 65536
> MAX_EFFECTIVE_BALANCE 32 ETH · EJECTION_BALANCE 16 ETH · SLOTS_PER_EPOCH 32

# CORE-CFG26 | Deployment-manifest drift across branches; production manifest set is not identified.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Deployment manifest integrity | Low | 3.3 | Repository-wide: core-block, docker-k8s-config and update-config manifest sets; genesis gas limit at k8s/values.yaml:60 and docker-compose/config/genesis/values.env.template:31 | Resolved |

BVSS 1.8 · Evidence grade E3

### Description

The repository carries three overlapping and mutually inconsistent deployment configurations with no marker identifying which is authoritative. The update-config branch retains a legacy configurations/ directory that exists nowhere else, and it diverges materially: a different chain ID, a weakened Eth1 follow distance, additional committed mnemonics, and the complete deletion of the crypto/dilithium package. Meanwhile docker-k8s-config lacks the Dilithium package and the Dilithium RPC block that core-block has. The consequence is that the answer to a basic due-diligence question — what is actually running in production — cannot be determined from the repository. This review treated core-block as the integration trunk based on branch topology and feature completeness, and the client should confirm that inference. A separate observation in the same area: the genesis gas limit is 60,000,000 on the trunk and 45,000,000 on update-config, against a typical Ethereum mainnet figure of roughly 30-36 million. A higher gas limit is a deliberate throughput choice rather than a security weakening, but it raises per-block execution cost, accelerates state growth, and increases the resource floor for anyone running a node; validating it against real node hardware and archive-storage projections is worthwhile before scaling. Container images are also pinned to floating :latest tags throughout, as noted in CORE-CFG07, which means the deployed software version is not reproducible from the repository at any commit. Nothing malicious was found in the deployment artefacts: the genesis premine file is an empty object, and the preloaded contract is the standard Ethereum deposit-contract bytecode, not a backdoor.

### Recommendation

Nominate one branch as the production trunk and state it in the repository README, then either merge the intended parts of update-config into it or delete that branch to remove the ambiguity — leaving a divergent branch carrying credentials and weakened parameters on a public-adjacent repository is a liability in itself. Pin every container image to an explicit tag and digest. Separate production manifests from devnet and reference tooling under distinct directories, so that the Kurtosis and test-fixture material cannot be mistaken for deployable configuration. Record the intended gas limit and the hardware profile it assumes in the operations documentation, and validate both against measured block-execution times.

### Mitigation / Status

RESOLVED 2026-08-03, verified on branch remediation/cfg-round1 (commit 46b7dfe). CFG Ninja independently verified this on the remediation/cfg-round1 branch (4 commits off dd105cb): the branch was fetched from origin and the diffs and build checked directly, not accepted on the commit hashes. The stale bootnode 18.134.110.61 is pruned from both lists in the committed production manifest — execution enodes 22 -> 21, beacon ENRs 22 -> 21, zero occurrences remain (CFG Ninja confirmed by decoding the committed deploy/production/docker-compose.yml). core-block is named as the production trunk in the README, and production and devnet manifests are now separated. The running fleet picks up the pruned bootnode list at the next image rollout; the pruned host is dead, so the interim exposure is negligible.

Previous: CONFIRMED INSTANCE 2026-08-03. CFG Ninja's passive decode of the production manifest found a single host, 18.134.110.61, present in BOTH the execution bootnode list and the beacon bootstrap list, advertising Lighthouse 8.0.1, on Krown mainnet (beacon fork digest 6d149307 matching the rest of the fleet), in an AWS eu-west-2 range — contradicting the client's stated uniform-Prysm fleet and no-AWS posture. The client verified it is a STALE entry, not a live node: TCP 3303 and 9000 time out from off-mesh (their live nodes refuse rather than drop), there is zero log contact with the address across the retained window on execution and consensus, and a live production beacon reports

8 connected / 56 disconnected peers with no Lighthouse in the connected set — the operating fleet is Prysm v7.1.3. It is a relic of the pre-migration AWS/Lighthouse deployment that survived into the current manifest unpruned; the client is removing it from both lists. This is the same root cause as CORE-CFG12 (repository and manifests not updated after the production migration) and is a concrete example of why production manifests must be treated as maintained artifacts — a stale bootnode is dialled indefinitely, and a reassigned IP could point new nodes at a stranger's host. Evidence is client-attested active verification, consistent with CFG Ninja's passive decode; independent confirmation would come from the pruned manifest or from active probing outside this engagement's scope.

Previous: Client confirmed 2026-08-02: core-block is the production trunk; update-config is legacy and is being retired rather than merged. Retiring it removes the divergent chain ID, the weakened Eth1 follow distance and the additional committed seeds in a single action.

Previous: No malicious or unexpected content found in the deployment artefacts. The finding concerns ambiguity and reproducibility, not a specific exploitable defect.

**References:** CWE-1188: Insecure Default Initialization of Resource | CWE-710: Improper Adherence to Coding Standards | CIS Docker / Kubernetes benchmark: image pinning

Evidence (reviewed source)

```
Branch divergence:
 update-config @ 4f6430b  legacy configurations/ dir; chain ID 89127398; ETH1_FOLLOW_DISTANCE=10;
                  extra committed mnemonics; crypto/dilithium DELETED (256 lines)
 docker-k8s-config @ 92ef598  no crypto/dilithium, no Dilithium RPC block (-70 lines in api.go)
 core-block @ dd105cb     full PQC feature set  <- treated as trunk in this review

Gas limit:
 k8s/values.yaml:60                          GENESIS_GASLIMIT=60000000
 docker-compose/config/genesis/values.env.template:31 GENESIS_GASLIMIT=60000000
 configurations/config/values.env:39 (update-config)  GENESIS_GASLIMIT=45000000

Reviewed and found benign: genesis premine file is empty ({}); preloaded contract is the
standard ETH2 deposit-contract bytecode.

Stale bootnode (2026-08-03), in both lists of the production manifest:
 18.134.110.61  el enode :3303  +  bn ENR :9000
 ENR advertises Lighthouse 8.0.1 (only 1 of 22; Prysm sets no client field)
 beacon fork digest 6d149307 = Krown mainnet (matches all 21 others)
 AWS eu-west-2 range
 Client verification: ports time out off-mesh; zero log contact; live beacon
 peer set 8 connected / 56 disconnected, no Lighthouse connected -> fleet is Prysm v7.1.3.
 Verdict: stale pre-migration relic, being pruned.
```

# CORE-CFG34 | Measured 0% test coverage on the rewritten handshake, the KEM wrapper and the PQC RPC endpoint.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Test coverage of new & modified code | Medium | 5.0 | p2p/rlpx/ (contains only buffer.go and rlpx.go); crypto/kyber/ (no test file); internal/ethapi/ (no test file); crypto/dilithium/ dilithium_test.go (sole test covering custom code); .github/ workflows/build-and-push-ecr.yml and dev-krown-node.yml | Out of scope (noted) |

BVSS N/A · Evidence grade E2

### Description

Coverage has now been measured rather than inferred. Running go test with a coverage profile restricted to the modification surface — the only measurement that means anything for a fork, since a tree-wide figure is dominated by inherited upstream tests — gives 29.4% of statements overall, and that figure is carried almost entirely by two packages the fork barely touched. The three packages containing this assessment's most serious findings are each at exactly 0.0%: crypto/kyber (the KEM on which all transport confidentiality now depends), p2p/rlpx (the rewritten handshake), and internal/ethapi (the Dilithium RPC endpoint). The function-level profile makes the connection to the other findings unambiguous. handleAuthMsg, which contains the discarded peer-identity check of CORE-CFG24, is at 0.0%. secrets, which contains the session-secret disclosure of CORE-CFG06, is at 0.0%. So are Handshake, runInitiator, runRecipient, makeAuthMsg, handleAuthResp and every other function in the handshake path. Not one line of the rewritten RLPx handshake is exercised by any test. By contrast crypto/dilithium reaches 60.0% and core/types 49.6%, which shows the team can and does write tests when it chooses to; the gap is specific to the security-critical rewrite, not general. p2p/rlpx contains only buffer.go and rlpx.go; upstream go-ethereum ships tests for this package, and they did not survive the rewrite. Consistent with CORE-CFG02, that upstream-deletion claim is stated from knowledge of the upstream project and will be diff-confirmed in the final report. Neither GitHub Actions workflow runs the suite. One builds and pushes a Docker image to ECR, the other sends a deploy script over SSM. There is no gate that would fail a merge on a broken test, and no coverage, static-analysis, vulnerability or secret scanning in CI either; a gap whose cost is now quantified by CORE-CFG01, where seven reachable dependency vulnerabilities were sitting undetected in a tree that deploys to production without executing an analyser.

### Recommendation

Treat test coverage of the modification surface as release-blocking, and measure it there rather than tree-wide, since the inherited upstream suite makes a tree-wide figure meaningless. Concretely: restore or rewrite the upstream p2p/rlpx tests against the new handshake and add cases that assert a handshake fails when the identity signature does not match the claimed public key, which directly regresses CORE-CFG24; add unit tests for crypto/kyber covering encapsulation and decapsulation round-trips, wire-size handling and behaviour when the security level changes; add tests for the Dilithium RPC endpoint including the negative case that a transaction with an invalid Dilithium signature is rejected; and add an end-to-end test that submits a type-0x75 transaction through the standard RPC and asserts pool admission and block inclusion, which regresses CORE-CFG04. Separately, add a CI workflow that runs go test ./..., reports coverage on the changed packages, and gates merges on both. Add gosec, staticcheck, govulncheck and secret scanning to the same pipeline, the last of which addresses the recurrence risk in CORE-CFG19. A codebase that deploys to production without executing its own tests has no automated defence against regression in exactly the cryptographic code this assessment is being asked to vouch for.

### Mitigation / Status

SCOPE POSITION, 2026-08-02. The client response of 2 August 2026 did not address this finding, and closed with "This covers everything from the protocol level: chain core, consensus and PQC. Anything else is out of scope."

CFG Ninja reads that statement as referring to other COMPONENTS — the wallet layer, the bridge, the DEX — all of which this assessment already excludes, rather than as declining this finding. This finding concerns the test coverage of p2p/rlpx, crypto/kyber, internal/ethapi and core/types: the precise packages the engagement was commissioned to

review, not a separate component.

We record the following against ourselves nonetheless. Engineering assurance was not named in the scope agreed at intake; the category was added to our framework on 1 August 2026, during this engagement. A client is entitled to decline a finding raised under a category they never agreed to, and if that is the client's position they have the scope document on their side. The finding is therefore retained on its facts — which are measured, and diff-confirmed against upstream v1.16.2 — but the client is NOT treated as owing a remediation response on it, and it is excluded from the "unresolved" pressure applied to the in-scope findings.

A direct scope question has been put to the client rather than the matter being assumed either way. If engineering assurance is accepted into scope for the final report, this finding proceeds normally; if it is declined, it will be recorded in the final report as raised, measured, and outside the agreed scope, with no remediation expectation attached.

Substantively the finding is unchanged and CFG Ninja does not withdraw it. The two functions containing CORE-CFG24 and CORE-CFG06 — handleAuthMsg and secrets — are at exactly 0.0% coverage, and the client's own root-cause account of CORE-CFG24 (the recovered key lost its consumer when derivation moved onto the Kyber shared secret) describes precisely the class of regression a handshake test detects.

Previous: Open. The inherited upstream suite provides genuine assurance for unmodified upstream code, which is the majority of the tree — but none for the custom work that is the subject of this assessment.

**References:** CWE-1120: Excessive Code Complexity (untested modification surface) | OWASP SCVS V1: verification of third-party and modified components | Go testing and coverage tooling: go test -coverprofile

Evidence (reviewed source)

```
go test -coverprofile -covermode=count  (Go 1.26.5, 2026-08-01, core-block @ dd105cbf)
  krown/crypto/kyber      coverage:  0.0% of statements   <- KEM, transport-critical
  krown/p2p/rlpx          coverage:  0.0% of statements   <- rewritten handshake
  krown/internal/ethapi   coverage:  0.0% of statements   <- Dilithium RPC endpoint
  krown/crypto/dilithium  coverage: 60.0% of statements
  krown/core/types        coverage: 49.6% of statements
  TOTAL (modification surface):     29.4% of statements

Function-level profile, p2p/rlpx — every handshake function unexercised:
  rlpx.go:285  Handshake      0.0%
  rlpx.go:413  runRecipient   0.0%
  rlpx.go:447  handleAuthMsg  0.0%  <- contains CORE-CFG24
  rlpx.go:501  secrets        0.0%  <- contains CORE-CFG06
  rlpx.go:557  runInitiator   0.0%
  rlpx.go:587  makeAuthMsg    0.0%
  rlpx.go:624  handleAuthResp 0.0%

CI (neither workflow executes tests or any analyser):
  .github/workflows/build-and-push-ecr.yml  build + push image to ECR
  .github/workflows/dev-krown-node.yml      deploy script via SSM
```

# CORE-CFG38 | Static analysis triage: 28 raw findings, no Krown-introduced defect confirmed.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Static analysis & tool triage | Informational | N/A | crypto/dilithium/dilithium.go:25; p2p/rlpx/ rlpx.go:332-333, :703, :230-243; internal/ethapi/*; core/types/bal/ bal_test.go:17 | Noted |

BVSS N/A · Evidence grade E2

### Description

gosec and staticcheck were run against the custom packages. This finding records the triage rather than the raw output, because an unfiltered tool dump would misrepresent the codebase. gosec reported 26 issues across 13 files. Twenty are G115 integer-conversion warnings, a rule that fires prolifically on stock go-ethereum and which the reviewed instances share with upstream. Two are G407 hardcoded-IV warnings at p2p/rlpx/rlpx.go:332-333 — these were read directly and are stock upstream code carrying the upstream rationale in a source comment: the all-zero IV is sound because AES-CTR is keyed with a fresh, single-use ephemeral session key, so IV reuse across sessions cannot occur. One is a G404 weak-random warning at rlpx.go:703, which is math/rand choosing only the length of EIP-8 handshake padding; the encryption on the following line draws from crypto/rand. Three are G104 unhandled-error warnings on hash writes, which cannot fail. Conclusion: no gosec finding was confirmed as a Krown-introduced defect. Definitive upstream-versus-fork attribution still depends on the mechanical diff in CORE-CFG02, and that qualification is deliberate. staticcheck returned two issues, one of which is worth the client's attention: crypto/dilithium/dilithium.go:25 declares var errNilPublicKey and never uses it (U1000). An error value defined for a nil-public-key condition, in a signature-verification package, that is never returned, indicates a guard that was written and never wired up. It is not exploitable as written, but it is exactly the kind of loose end that the missing tests in CORE-CFG34 would have surfaced. The other is a cosmetic S1008 simplification in an inherited upstream test file. The tree builds clean — go build ./... exits 0; which is what makes the proof-of-concept work listed in section 9 feasible for the final report.

### Recommendation

Wire up or delete errNilPublicKey in crypto/dilithium/dilithium.go, and add the nil-key test case that would have caught it. Add gosec and staticcheck to CI alongside govulncheck, with a documented baseline suppressing the inherited upstream G115 class so that the signal from newly-written code is not lost in it — an unfiltered gosec gate on a go-ethereum fork will be ignored within a week, which is worse than no gate. Re-run all three analysers after the mechanical fork diff, so that any future finding can be attributed to Krown code with certainty rather than judgement.

### Mitigation / Status

Not applicable: informational record of the analysis performed and the reasoning applied to it.

**References:** gosec (securego) | staticcheck (honnef.co/go/tools) | CWE-1164: Irrelevant Code

### Evidence (reviewed source)

```
gosec v2.28.0 — custom packages only (13 files, 4,934 lines): 26 issues
  20x G115  integer overflow conversion      internal/ethapi/*, p2p/rlpx/*   inherited upstream pattern
   2x G407  hardcoded IV                     p2p/rlpx/rlpx.go:332-333       stock upstream; see below
   1x G404  weak RNG (math/rand)             p2p/rlpx/rlpx.go:703           padding length only
   3x G104  unhandled errors                 p2p/rlpx/rlpx.go:230,235,243   hash writes; cannot fail

rlpx.go:328-333 — the upstream rationale, read at source:
    // we use an all-zeroes IV for AES because the key used
    // for encryption is ephemeral.
    iv := make([]byte, encc.BlockSize())

rlpx.go:701-708 — math/rand chooses padding length; crypto/rand does the encryption:
    h.wbuf.appendZero(mrand.Intn(100) + 100)
    enc, err := ecies.Encrypt(rand.Reader, h.remote, h.wbuf.data, nil, prefix)

staticcheck 2026.1 — 2 issues:
  crypto/dilithium/dilithium.go:25  var errNilPublicKey is unused (U1000)   <- unwired guard
  core/types/bal/bal_test.go:17     S1008 simplification (inherited upstream test)
```

# CORE-CFG01 | Seven reachable vulnerabilities in pinned dependencies, including the PQC library and the RPC auth handler.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Upstream baseline & version currency | Medium | 5.9 | go.mod; reachable traces into crypto/kyber/kyber.go:184 and :255, node/jwt_handler.go:44, crypto/kzg4844/kzg4844_gokzg.go:26, p2p/nat/stun.go:13, accounts/scwallet/securechannel.go:68, cmd/devp2p/dns_cloudflare.go:61 | Resolved (fix verified in source; deployment pending) |

BVSS 4.8 · Evidence grade E2

### Description

The upstream baseline is go-ethereum v1.16.2 stable, confirmed from the version package. The Go module was renamed from github.com/ethereum/go-ethereum to krown with import paths rewritten throughout, which matters for a reason beyond cosmetics: advisory tooling that resolves by module path will not automatically match this tree against go-ethereum advisories, so upstream CVE tracking has to be deliberate rather than assumed. This item is no longer pending. A govulncheck run against the tree reports that the code is affected by SEVEN vulnerabilities across seven modules, each with a reachable call trace from Krown's own code. Two are materially relevant to a production node. First, GO-2026-4550 affects github.com/cloudflare/circl v1.6.1 — the exact library and version providing every post-quantum primitive in this fork — and the reachable traces run directly through Krown's own Kyber wrapper at crypto/kyber/kyber.go:184 (Encapsulate) and :255 (Decapsulate). It is fixed in CIRCL v1.6.3. The client has since established, and CFG Ninja accepts, that GO-2026-4550 concerns an incorrect calculation in secp384r1 CombinedMult — a code path the Kyber wrapper does not enter. govulncheck reports symbol reachability, not path exploitability, and on this item the distinction matters. The CIRCL advisory is therefore recorded as informational within this finding rather than as an exploitable defect; the version bump is being applied regardless, which is correct practice for a chain marketed on its post-quantum properties. Second, GO-2025-3553 affects github.com/golang-jwt/jwt v4.5.1 with excessive memory allocation during header parsing, reachable from node/jwt_handler.go:44 in the authenticated-RPC ServeHTTP path. That is remote, pre-authentication input reaching an allocation defect; the most operationally exposed item in the set. Fixed in v4.5.2. The remaining five are lower relevance but real: golang.org/x/text v0.23.0 (infinite loop on invalid input, fixed v0.39.0), golang.org/x/net v0.38.0 (idna Punycode handling, fixed v0.55.0), github.com/consensys/gnark-crypto v0.18.0 (unchecked allocation during deserialisation, reachable via KZG init, fixed v0.18.1), github.com/pion/dtls v2.2.7 (AES-GCM nonce handling, no fix available, reached via NAT/STUN init), and golang.org/x/crypto v0.36.0 (the unmaintained openpgp package, reached only from build tooling, no fix available). Read together with CORE-CFG34, this is the predictable consequence of a pipeline that never runs an analyser: none of these would have survived a CI job that executes govulncheck.

### Recommendation

Raise CIRCL to v1.6.3 or later and golang-jwt to v4.5.2 or later as immediate, low-risk dependency bumps; both are patch-level and neither should require code changes. Then update x/text to v0.39.0, x/net to v0.55.0 and gnark-crypto to v0.18.1. For pion/dtls and x/crypto/openpgp, where no fix exists, confirm the reachability is genuinely confined to NAT discovery and release tooling respectively, and document the accepted risk. Separately from the individual bumps, add govulncheck to CI as a merge gate — this is the single highest-value change available here, because it converts an ongoing manual obligation into an automatic one. Finally, review the go-ethereum security advisories published after v1.16.2 and apply any security-relevant upstream fix; the module rename means this will never happen automatically.

### Mitigation / Status

FIX VERIFIED IN SOURCE 2026-08-03, deployment pending. On branch remediation/cfg-round1 (commit 5e73afe) go.mod raises CIRCL v1.6.1->v1.6.3, golang-jwt/jwt/v4 v4.5.1->v4.5.2, x/text v0.23.0->v0.39.0, x/net v0.38.0->v0.56.0, gnark-crypto v0.18.0->v0.18.1 (go directive 1.23.0->1.25.0 as a transitive requirement). CFG Ninja re-ran govulncheck on the branch (Go 1.26.5) and independently confirms 7 vulnerabilities -> 2. The two remaining are the ones with no upstream fix — GO-2026-5932 (x/crypto/openpgp, reached only from internal/build PGP signing tooling) and GO-2026-4479 (pion/dtls, reached via p2p/nat STUN init) — and are recorded as accepted residuals. The reachable,

exploitable driver of this finding, GO-2025-3553 (golang-jwt allocation via the RPC auth handler), is fixed. This also settles the earlier CIRCL question: the GO-2026-4550 traces from the Kyber wrapper terminate in Kyber lattice operations and do not reach secp384r1 CombinedMult, confirming module-linkage rather than path reachability — the client applied the bump regardless. NOT YET RESOLVED because the live fleet still runs commit dd105cbf (confirmed via web3_clientVersion) with the old dependency set; it clears on the scheduled image rollout, at which point CFG Ninja will re-verify against the live build.

Previous: Severity held at Medium on the strength of GO-2025-3553 (golang-jwt v4.5.1) alone — remote, pre-authentication input reaching an allocation defect in the RPC authentication handler. The CIRCL item is downgraded to informational following the client's correct observation about path reachability. Client is applying all version bumps.

**References:** go-ethereum security advisories | golang.org/x/vuln/cmd/govulncheck | CWE-1104: Use of Unmaintained Third Party Components | CWE-1395: Dependency on Vulnerable Third-Party Component

Evidence (reviewed source)

```
govulncheck ./...  (Go 1.26.5, run 2026-08-01 against core-block @ dd105cbf)
 "Your code is affected by 7 vulnerabilities from 7 modules."

 GO-2026-4550  cloudflare/circl     v1.6.1  -> fixed v1.6.3
          reachable: crypto/kyber/kyber.go:184 Encapsulate, :255 Decapsulate
 GO-2025-3553  golang-jwt/jwt/v4    v4.5.1  -> fixed v4.5.2
          reachable: node/jwt_handler.go:44 jwtHandler.ServeHTTP -> ParseWithClaims
 GO-2026-5970  golang.org/x/text    v0.23.0 -> fixed v0.39.0
 GO-2026-5026  golang.org/x/net     v0.38.0 -> fixed v0.55.0
 GO-2025-4087  consensys/gnark-crypto v0.18.0 -> fixed v0.18.1  (via kzg4844 init)
 GO-2026-4479  pion/dtls/v2         v2.2.7  -> no fix        (via p2p/nat/stun.go:13)
 GO-2026-5932  golang.org/x/crypto  v0.36.0 -> no fix        (openpgp, build tooling only)

The tree builds clean: go build ./... exits 0.
```

# CORE-CFG02 | Modification surface confirmed by mechanical fork diff against upstream v1.16.2.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Modification surface | Informational | N/A | Repository-wide; git history (169 commits, squashed re-import beginning Initial commit -> migrate repo) | Resolved |

BVSS N/A · Evidence grade E2

### Description

The modification surface has now been established mechanically, closing the limitation this finding recorded. CFG Ninja vendored a clean upstream go-ethereum v1.16.2 tree, normalised the module-path rename (krown -> github.com/ethereum/go-ethereum) and stripped licence headers and comments, then compared executable code file by file across the consensus, state-transition, sync, block-validation, EVM and txpool surface.

Result: every consensus/state/sync-critical file is code-identical to upstream v1.16.2. Of fourteen such files checked, twelve are byte-identical after normalisation; the two that flagged differ only by the module rename — core/txpool/legacypool/legacypool.go is set-identical (a line-ordering artifact), and eth/downloader/downloader.go differs solely in the type reference ethereum.SyncProgress vs krown_blockchain.SyncProgress, the same type under a renamed import. The fork strips upstream licence headers but does not alter executable code in any of these paths.

The modification surface is therefore confined to exactly the files identified by source reading: core/types/transaction.go (about 156 custom code lines, the PQC tx type), core/types/transaction_signing.go (about 19), the new packages crypto/dilithium and crypto/kyber, p2p/rlpx/rlpx.go (about 44, the handshake rewrite), and internal/ethapi/api.go (about 31, the Dilithium RPC). No consensus, state-transition, sync or block-validation path is modified. This confirmation is the evidentiary basis for the state-management scope position in section 12: those paths are stock go-ethereum v1.16.2.

### Recommendation

None outstanding for this item. Maintaining the fork with a real upstream remote and merge history (rather than the current squashed re-import) would keep future diffs cheap and is recommended as good practice, not as a finding.

### Mitigation / Status

Resolved 2026-08-04. Mechanical diff performed and the modification surface confirmed; no consensus/state/sync-critical file is altered at the code level.

**References:** CWE-1357: Reliance on Insufficiently Trustworthy Component | OWASP SCVS: component provenance

Evidence (reviewed source)

```
Mechanical fork diff, Krown core-block (dd105cb) vs upstream go-ethereum v1.16.2, code-only:
 CONSENSUS / STATE / SYNC — all stock:
   state_transition, state_processor, block_validator, blockchain, chain_makers,
   consensus/beacon, eip1559, txpool/validation, vm/interpreter, vm/contracts,
   downloader (module-alias only), skeleton, legacypool (set-identical),
   params/protocol_params  -> CODE IDENTICAL
 MODIFICATION SURFACE (custom):
   core/types/transaction.go (+156), transaction_signing.go (+19),
   crypto/dilithium/* (new), crypto/kyber/* (new),
   p2p/rlpx/rlpx.go (+44), internal/ethapi/api.go (+31)
```

# CORE-CFG03 | Custom transaction type 0x75: mechanics and naming.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Custom transaction types | Informational | N/A | core/types/transaction.go:41, :203, :761, :871-891; core/types/transaction_signing.go:207-209, :248-250, :456-468, :485 | Noted |

BVSS N/A · Evidence grade E3

### Description

This finding documents how transaction type 0x75 actually works, since three separate findings depend on it and the name invites a reading the code does not support. The type is registered as a decodable EIP-2718 typed envelope in the same way as any other type, and it is marked as a known type to the signer with identical treatment to the dynamic-fee, blob and set-code types. Two things differ from a standard transaction: the signature hash is computed with SHAKE256-256 rather than Keccak-256, and the sender address is derived with SHAKE256 rather than Keccak-256. Everything else is unchanged. The signature values remain plain V, R and S; there is no Dilithium signature field on the type; sender recovery goes through crypto.Ecrecover, which is secp256k1; and Transaction.Hash(), the transaction ID, remains Keccak-256 via the normal code paths. The consequence worth stating plainly is that a transaction of type 0x75 is not post-quantum in any respect — it is an ordinary ECDSA transaction whose signing digest happens to be computed with a different hash function. Changing the hash over which a signature is computed does not confer any property of the hash on the signature scheme. If ECDSA is broken, this type offers no additional protection whatsoever. The naming is therefore likely to mislead users, integrators and counterparties, which is why it is reflected in CORE-CFG18. Two further consequences are recorded separately: the address-derivation branch creates the dual-address hazard of CORE-CFG15, and the type cannot presently be admitted by the transaction pool per CORE-CFG04.

### Recommendation

Rename the type and every associated identifier so the name describes what it is — a SHAKE256 signing-domain variant — rather than implying post-quantum signature security, and correct the documentation accordingly. If a genuinely post-quantum transaction type is the intent, see the recommendation in CORE-CFG14: it requires a signature field carried in the transaction encoding and verified in the state transition. Retain the intended purpose of the change if there is one, SHAKE256 does provide clean domain separation between transaction types; but do not let domain separation be presented as quantum resistance.

### Mitigation / Status

Not applicable: informational record of implemented behaviour, provided as the basis for CORE-CFG04, CORE-CFG15 and CORE-CFG18.

**References:** EIP-2718: Typed Transaction Envelope | EIP-155: Simple Replay Attack Protection | NIST FIPS 202 (SHA-3 / SHAKE)

Evidence (reviewed source)

```
core/types/transaction.go:41    PQCTxType = 0x75
core/types/transaction.go:203   case PQCTxType: inner = new(TxDataPQC)  // envelope registration only
core/types/transaction.go:761   rawSignatureValues() -> plain V, R, S *big.Int (no Dilithium field)
core/types/transaction.go:874-891 sigHash() uses xsha3.NewShake256(); source comment notes the
                                  txID (Transaction.Hash) remains Keccak-256
core/types/transaction_signing.go:209  s.txtypes[PQCTxType] = struct{}{}  // known-type marker only
core/types/transaction_signing.go:485  recoverPlain() -> crypto.Ecrecover()  // secp256k1
```

# CORE-CFG17 | Cryptographic library sourcing and pinning.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Crypto library sourcing & pinning | Informational | N/A | go.mod:96 (github.com/cloudflare/circl v1.6.1); crypto/dilithium/dilithium.go:9; crypto/kyber/kyber.go:9-11; core/types/transaction.go:12 (golang.org/x/crypto/sha3) | Noted |

BVSS 3.0 · Evidence grade E2

### Description

Recorded as a positive result with a caveat. All post-quantum primitives come from Cloudflare CIRCL v1.6.1, a well-regarded, actively maintained and independently reviewed open-source library, and the SHAKE256 implementation is from golang.org/x/crypto/sha3. No hand-rolled or proprietary cryptography was found anywhere in the fork: the client did not implement lattice cryptography itself, which is the single most common and most damaging mistake in this space, and that decision deserves credit. The caveat is one of selection rather than sourcing. Within CIRCL, the pre-standard round-3 packages were chosen over the FIPS-standardised ones that the same pinned version also provides — sign/dilithium/mode2 rather than sign/mldsa, and kem/kyber/kyber768 rather than kem/mlkem/mlkem768 — and within Dilithium the weakest of three parameter tiers was selected. Those choices are the subject of CORE-CFG13 and CORE-CFG16 and are not restated here. The dependency is pinned to an exact version in go.mod, which is correct practice; dependency-advisory scanning has now been run, and it changes the picture: CIRCL v1.6.1 is subject to GO-2026-4550, fixed in v1.6.3, with reachable traces through this fork's own Kyber wrapper. The pin is correct practice; the pinned version is out of date. See CORE-CFG01.

### Recommendation

Maintain the exact-version pin and add CIRCL to whatever dependency-monitoring process the team runs, so that advisories affecting it are noticed. When migrating to the FIPS-standardised primitives per CORE-CFG13 and CORE-CFG16, stay within CIRCL rather than introducing a second cryptographic library. Continue to avoid implementing post-quantum primitives in-house under all circumstances.

### Mitigation / Status

Library sourcing is sound. The pinned version carries an open advisory (CORE-CFG01) and the parameter/standard selection is tracked in CORE-CFG13 and CORE-CFG16.

**References:** Cloudflare CIRCL | NIST IR 8547 | OWASP SCVS: third-party component integrity

Evidence (reviewed source)

```
go.mod:96                  github.com/cloudflare/circl v1.6.1   (exact pin)
crypto/dilithium/dilithium.go:9  circl/sign/dilithium/mode2     (round-3; sign/mldsa available, unused)
crypto/kyber/kyber.go:9-11       circl/kem/kyber/kyber768       (round-3; kem/mlkem available, unused)
core/types/transaction.go:12     golang.org/x/crypto/sha3       (SHAKE256)

No hand-rolled or proprietary cryptographic implementation found anywhere in the fork.
```

# CORE-CFG28 | Genesis supply and validator-admission control concentrated in a single key.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Genesis allocation, supply & governance | Medium | 5.0 | Production genesis (A4, 2026-08-03): 0x13eceac6be81b2fc12d91efeb328012c0abfe306; deposit-contract allowlist controller | Acknowledged |

BVSS 4.5 · Evidence grade E2

### Description

Established from the production genesis supplied on 3 August 2026. The genesis allocation has 262 entries, of which 256 are standard 1-wei precompile seeding (0x00..00 through 0x00..ff) and one is a funded account, 0x13eceac6be81b2fc12d91efeb328012c0abfe306, holding the entire initial supply. No other funded accounts and no preloaded code beyond the standard deposit contract — no hidden mint, which is the reassuring part.

The finding is concentration, not misallocation. The same key that holds the full genesis supply is also the deposit-contract allowlist controller — the authority that decides which validators may join. Two distinct powers, economic ownership of the supply and admission control over the validator set, are vested in one key with no timelock or multisig disclosed. The client identifies the concentration themselves and ties it to the decentralisation roadmap, which is the correct framing for the present stage. It is recorded because a single key that can both move the supply and decide who validates is a single point of compromise that an exchange must weigh, and because separating those two powers is materially easier before external operators arrive than after.

### Recommendation

Separate the two authorities: the supply-holding account and the deposit-allowlist controller should not be the same key, and neither should be a single EOA. Place the allowlist controller behind a multisig or timelock ahead of Milestone 1, since it gates validator admission for the whole external-operator programme. Disclose the custody arrangement for the supply-holding key to BloFin explicitly rather than leaving it to be inferred from the genesis. Where on-chain vesting (the UNCX arrangement described to BloFin) governs the supply, confirm it on-chain in the final report.

### Mitigation / Status

Genesis hash independently verified by CFG Ninja 2026-08-03: eth_getBlockByNumber(0) on the live chain returns 0xd7b15bd8886e90ed7e66993c95bdb0b1489c1a965dccbdf3ee598399bb352f74, an exact match to the value Krown provided to BloFin, with genesis gas limit 60,000,000 and timestamp 2026-01-03T00:49:27Z. No hidden mint and no preloaded code beyond the standard deposit contract were found in the supplied genesis. The finding is not the allocation itself but the concentration of supply and validator-admission control in one key; that concern stands and is acknowledged by the client, tied to the decentralisation roadmap.

Previous: Open. Disclosed by the client and consistent with a single-operator MVP, but the concentration of two distinct powers in one key should be resolved before the validator set opens.

**References:** CWE-269: Improper Privilege Management | CWE-732: Incorrect Permission Assignment for Critical Resource

Evidence (reviewed source)

> Production genesis, client-supplied 2026-08-03:
> 262 allocations; 256 x 1-wei precompile seeding (0x00..00 - 0x00..ff)
> 1 funded account: 0x13eceac6be81b2fc12d91efeb328012c0abfe306 — full supply
> also the deposit-contract allowlist controller (client statement)
> no preloaded code beyond the standard deposit contract
> genesis hash 0xd7b15bd8886e90ed7e66993c95bdb0b1489c1a965dccbdf3ee598399bb352f74 (matches BloFin)

# CORE-CFG27 | Chain-level alerting not yet in place, and the exchange was told otherwise.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Monitoring, alerting & DR | Low | 3.1 | Client Q24 (2026-08-03) vs BloFin due-diligence response, operations section | Acknowledged |

BVSS N/A · Evidence grade E3

### Description

The client's Q24 answer is candid: Prometheus, Grafana and node exporters are live across the fleet for host-level metrics, but alerting on the chain-level signals — finality progression, reorg depth, and the latest-versus-finalized gap — is "the layer currently being built out." The due-diligence response provided to BloFin describes 24-hour alerting on those exact three signals as though it were in place.

The operational gap itself is Low for a single-operator MVP: with one operator watching the fleet, the absence of automated finality alerting is a resilience weakness rather than an active exposure. It becomes materially more important at the external-operator milestone, when no single party is watching everything. The reason it is raised at all, rather than left as an observation, is the representation mismatch: it is another instance of the BloFin materials describing a capability that is roadmap rather than production, and it belongs with CORE-CFG18.

### Recommendation

Complete the finality/reorg/latest-vs-finalized alerting and the associated runbooks, and prioritise it ahead of Milestone 2 when external operators join. Until it is in place, correct the BloFin response to describe monitoring accurately — host-level metrics live, chain-level alerting in progress. Provide the runbook and alert definitions to CFG Ninja when they exist so the DR posture can be assessed for the final report.

### Mitigation / Status

Open. Host-level monitoring is live; chain-level alerting is in progress. The representation gap is corrected in the same pass as CORE-CFG18.

**References:** CWE-778: Insufficient Logging | CWE-1053: Missing Documentation for Design

Evidence (reviewed source)

> Client Q24, 2026-08-03: Prometheus + Grafana + node exporters live (host metrics);
> chain-level alerting on finality / reorg depth / latest-vs-finalized "currently being built out".
> BloFin DD response: "24-hour alerting on finality progression, reorg depth, and latest-vs-finalized gap."

# CORE-CFG09 | Checkpoint sync and weak-subjectivity trust concentrated in a single internal endpoint.

| Category | Severity | CVSS | Location | Status |
|---|---|---|---|---|
| Fork choice, finality & sync | Low | 3.3 | docker-compose.yml:100-101 (--checkpoint-sync-url, --genesis-beacon-api-url); :77 (--min-sync-peers=1) | Detected |

BVSS 2.6 · Evidence grade E3

### Description

Every production node in the supplied manifest carries --checkpoint-sync-url and --genesis-beacon-api-url pointing at one internal host, and runs with --min-sync-peers 1. Weak-subjectivity trust is therefore concentrated in a single endpoint: a node bootstrapping from a compromised, stale or incorrect checkpoint source has no independent second opinion, and --min-sync-peers 1 means it will accept a chain view from a single peer. For a single-operator fleet syncing from the operator's own infrastructure this is low impact today — the operator already controls the nodes. It becomes a genuine question at the external-operator milestone, when independent operators bootstrap their nodes and the checkpoint source becomes a trusted third party they did not choose. A malicious or misconfigured checkpoint endpoint could sync a new operator onto a wrong view of finality.

### Recommendation

Before external operators onboard, document a weak-subjectivity checkpoint distribution model that does not rely on a single Krown-controlled endpoint — for example publishing signed weak-subjectivity checkpoints operators can verify against multiple sources — and raise --min-sync-peers above 1 for externally-operated nodes. For the current single-operator fleet, no change is required beyond documenting the trust assumption.

### Mitigation / Status

Low today under single-operator control. Flagged as a prerequisite to design before the external-operator programme, not an active exposure.

**References:** Ethereum weak-subjectivity documentation | CWE-1104: Use of Unmaintained/Single-Source Trusted Component

Evidence (reviewed source)

```
docker-compose.yml (production manifest):
 :100 --checkpoint-sync-url=http://<REDACTED-internal-host>:5052
 :101 --genesis-beacon-api-url=http://<REDACTED-internal-host>:5052
 :77  --min-sync-peers=1
```

# 9. Privileged Roles & Centralization

The network is operated as a fully permissioned system at the time of review. This is a legitimate posture for a chain at this stage, and it is currently the mitigating factor that keeps two transport-layer findings at High rather than Critical. It is recorded here because an exchange assessing listing risk needs an accurate picture of who can affect the network, and because the same posture is what will change at the decentralisation milestone.

| Role / Authority | Held by | Risk |
|---|---|---|
| Validator set | Krown Network (single operator) | Complete control over block production, transaction ordering and finality. A single entity can halt the chain, censor transactions, or finalise a chosen history. No external validator provides an independent check. This also means transaction ordering is entirely at the operator's discretion. |
| Consensus / execution client upgrade | Krown Network engineering | Consensus rules can be changed by deploying new node software to the fleet. With a single operator running every node, there is no timelock, no multisig and no social-consensus barrier between a code change and a live consensus change. No governance or upgrade-authority control was found in the repository. |
| Genesis and chain configuration | Krown Network engineering | Chain ID, gas limit, fork schedule and beacon preset are set from repository configuration. Reviewed content is benign — the premine file is empty and the preloaded contract is the standard deposit contract — but the values are unilaterally controlled and, per CORE-CFG26, the authoritative manifest set is not identified. |
| RPC node operation | Krown Network | All public RPC access is served by operator-run infrastructure. The node-local contract-deployment gate (CORE-CFG23) lets the operator refuse contract deployments through its own endpoints. Since the operator runs all endpoints, a network-level restriction is achievable in practice even though the flag itself is not a consensus rule. |

| Role / Authority | Held by | Risk |
|---|---|---|
| Validator key custody | Krown Network (not verified) | Key custody, keystore isolation and signing/withdrawal separation could not be verified from source and are deferred to the final report. Committed test credentials are addressed in CORE-CFG19; that finding does not establish anything about production key handling, which remains unassessed. |
| P2P peer admission | Krown Network | The permissioned peer set is what currently bounds CORE-CFG06 and CORE-CFG24. Both findings escalate toward Critical when external peers are admitted, so peer admission is a security-relevant control today and its removal is a security event, not merely a decentralisation milestone. |

# 10. Potential Risks (Systemic)

These are design-level / operational risks rather than isolated code defects. They are disclosed transparently and factored into the overall posture.

## Single-operator control of consensus

Every validator is operated by Krown Network. The chain therefore offers no censorship resistance and no liveness guarantee independent of the operator, and finality reflects a single entity's decisions. This is disclosed and is a normal stage for a young network, so it is not scored as a code defect. It is recorded because an exchange must understand that finality assurances on this chain currently rest on the operator's continued honest operation and infrastructure availability, not on distributed consensus. Deposit-crediting policy should be set with that in mind.

## Findings that escalate at the decentralisation milestone

CORE-CFG06 (session secret written to stdout) and CORE-CFG24 (peer identity check discarded) are rated High at present-day impact because the peer set is closed and operator-controlled. Both become substantially more serious the moment external validators or peers are admitted: the first exposes third-party session secrets in operator logs, and the second permits peer-identity spoofing by parties outside the operator's control. Both should be treated as blocking prerequisites for that milestone rather than as items to remediate afterwards. This report states the escalation explicitly rather than folding it into the score, so that neither the client nor the exchange is surprised by it later.

## Post-quantum positioning versus implemented reality

The network is presented on the strength of its post-quantum properties, and one of them is real: RLPx transport confidentiality is protected by a mandatory Kyber768 KEM. The signature layer, which is what protects on-chain value, is not post-quantum at all, and both primitives are pre-standard round-3 schemes rather than the finalised NIST standards. The systemic risk is reputational and commercial rather than technical: if the network is listed and marketed on a security property it does not have, the correction is more damaging later than now. Aligning the public claims with the implementation, or completing the implementation to match the claims, is a decision the client should take before listing rather than after.

## Development-stage maturity signals

Several independent observations point the same way: debug prints left in cryptographic and signing hot paths, stock devnet template values never customised including the chain ID, credentials intentionally committed to the repository, three inconsistent deployment manifest sets with no authoritative one identified, floating :latest image tags, a squashed re-import history without upstream merge lineage, and a post-quantum feature that exists on one branch and is deleted on another. Individually each is minor. Together they indicate a codebase at an internal-development stage rather than one hardened for production custody of third-party value, and the remediation effort should be scoped accordingly.

## Wire-format incompatibility with standard Ethereum clients

Because the RLPx handshake was rewritten with mandatory Kyber fields and no capability negotiation, a stock go-ethereum node cannot complete a handshake with a Krown node. This is a deliberate consequence of the design rather than a defect, but it has practical reach: standard tooling, third-party indexers, monitoring agents and bridge clients that expect to speak devp2p will not connect, and any integrator planning to run their own node must use Krown's client build. Exchanges and infrastructure providers should factor this into integration effort estimates, and it means the operator cannot fall back to a stock client in an incident.

## Migration hygiene: the repository was never updated after the production migration

Four findings in this report share a single root cause, and are most efficiently closed as one piece of work rather than four. Krown migrated its production deployment off AWS and onto Prysm following the May 2026 incident, but the repository and the deployment manifests were not updated to match. The consequences surface separately: the repository configures Lighthouse where production runs Prysm (CORE-CFG07); no production chain configuration exists in version control at all, so the deployed network is not reproducible from the repository (CORE-CFG12); the manifests carry devnet chain IDs and a weakened follow distance that are not the production values (CORE-CFG08, CORE-CFG12); and a stale pre-migration bootnode survived into the current production manifest, advertising the old Lighthouse client on an AWS host that no longer runs (CORE-CFG26). None of these is a code defect. All four are the same gap between what runs and what is written down.

Consolidated remediation, one workstream: bring the production configuration into version control — the manifest structure with secrets injected at deploy time, plus a Krown-specific chain-config object in params/config.go carrying the real chain ID and follow distance; reconcile the consensus-client identity to Prysm v7.1.3 throughout; prune the stale bootnode from both the execution and beacon lists; and retire the update-config branch. Doing this once removes a whole class of "the repository does not match production" findings, and it is the single most valuable pre-listing cleanup available because it is what lets BloFin — or any third party — verify the live network against something committed. The client has identified this root cause themselves and committed to the fix ahead of the external-operator programme.

## Withdrawn: replay exposure from a shared devnet chain ID

This item appeared in an earlier revision of this report issued on 1 August 2026 and is WITHDRAWN IN FULL. It asserted cross-network replay exposure on the basis of the chain ID configured in the repository. That was our error: production carries --networkid=1983 and DEPOSIT_CHAIN_ID 1983, matching the value CFG Ninja verified live in section 6, and Krown is registered in the ethereum-lists/chains registry as eip155:1983. The 3151908 value is test configuration only. A reader of the withdrawn item alone would have taken away the opposite of section 6 — an internal contradiction in our own report.

The withdrawal is recorded rather than removed, in keeping with the evidence standards in Appendix C. It is the second occasion on which a repository-only view produced a wrong conclusion about chain identity, the first being the original draft of CORE-CFG12. Both are retained as a stated limit of source-only analysis.

# 11. Verification Performed & Re-verification Schedule

Independent verification completed during this assessment. Each was executed by CFG Ninja, not taken on the client's representation.

## Completed

- Independent verification of the round-1 remediation branch (diffs checked, govulncheck re-run 7->2).
- Mechanical fork diff vs upstream go-ethereum v1.16.2 — modification surface confirmed, consensus/state/sync stock (CORE-CFG02).
- Live-network verification: chain identity, production build, finality tags, RPC namespace posture, and genesis hash (exact match) — all confirmed against mainnet.
- Consensus parameters read from the live beacon /config/spec and reconciled against the mainnet preset — no weakening (CORE-CFG07/CFG08).
- Committed-secret enumeration across all branches and full history; the seventh mnemonic derived and cleared (CORE-CFG19).

## Remaining — CFG Ninja re-verifies each before it is reported as closed

- DEPLOYMENT re-verification: once the fixed build is rolled out to the fleet, confirm the live web3_clientVersion reports the new commit and that the CORE-CFG06 session-secret log purge has been completed. CFG Ninja issues a short deployment-verification addendum on this.
- Confirm the corrected due-diligence text has been delivered to BloFin (CORE-CFG18 — wording already verified accurate against source).
- Re-verify the stage-gated remediations at source when they land, ahead of external operators: CORE-CFG08 (Eth1 follow distance to 2048), CORE-CFG24 (handshake proof-of-possession), CORE-CFG14/CFG16 (consensus-level PQC enforcement and hybrid KEM).
- Address the one open low-severity finding, CORE-CFG09 (checkpoint-sync trust), before external operators.
- If a runnable node is later provided, exercise state-management and sync runtime behaviour to close the remaining scope limitation (the mechanical diff confirms these paths are stock upstream, so residual risk is assessed low).

# 12. Assessment Summary

| Metric | Value |
|---|---|
| Total findings | 24 |
| Production build verified | YES — live mainnet reports Geth/v1.16.2-stable-dd105cbf, matching the reviewed commit exactly |
| Critical | 0 |
| High | 0 — none open. CORE-CFG24 is reported scope-limited at Medium (source defect in scope; exploitability out of scope). One finding, CORE-CFG24, escalates toward High/Critical at the external-validator milestone and is being fixed ahead of it. |
| Medium | 13 — 7 resolved, 5 acknowledged, 1 out of scope |
| Low | 6 — 1 resolved, 4 acknowledged, 1 open |
| Informational | 5 — 2 resolved, 3 noted (scored at zero) |
| Resolved | 10 — CORE-CFG01, CORE-CFG02, CORE-CFG06, CORE-CFG07, CORE-CFG10, CORE-CFG12, CORE-CFG18, CORE-CFG19, CORE-CFG23, CORE-CFG26. CORE-CFG01 and CORE-CFG06 are fixed and verified in source with deployment to the live fleet pending. |
| Acknowledged (committed, not yet re-verified) | 9 — client accepted and committed to remediation; CFG Ninja re-verifies each when the fix lands. CORE-CFG24, CORE-CFG14, CORE-CFG13, CORE-CFG16, CORE-CFG15, CORE-CFG04, CORE-CFG08, CORE-CFG28, CORE-CFG27. |
| Open | 1 — CORE-CFG09 (Low). A low-severity design note, addressed before external operators. |
| Informational / out of scope (no remediation) | 4 — CORE-CFG34, CORE-CFG38, CORE-CFG03, CORE-CFG17 (scored zero or outside the agreed scope) |
| Not resolved (total) | 14 = 9 acknowledged + 4 informational/out-of-scope + 1 open |
| Dependency vulnerabilities | 7 reachable identified; reduced to 2 after the round-1 fix (CIRCL and golang-jwt bumps), both remaining having no upstream fix. CFG Ninja re-ran govulncheck to confirm (CORE-CFG01). |
| Test coverage of custom code | Measured 29.4% on the modification surface; 0.0% for p2p/rlpx, crypto/kyber and internal/ethapi (CORE-CFG34, out of scope) |
| Modification surface | Confirmed by mechanical diff vs upstream v1.16.2 — consensus/state/sync stock; custom code confined to 6 PQC/handshake/RPC files (CORE-CFG02) |

| Metric | Value |
|---|---|
| Evidence grades | E1 demonstrated: 0 · E2 tool-confirmed: 12 · E3 source-confirmed: 12 · E4 inferred: 0 |
| Post-Quantum Readiness Grade | PQR-2 (Experimental) — HNDL exposure: HIGH |
| Branches reviewed | 8 of 8 remote branches |
| Commits scanned (secrets, full history) | 169 |
| Reviewed commit | dd105cbf6d7ab0ceeba955fe489f92701bd430bd (core-block) — confirmed as the live production build |
| Round-1 remediation | Branch remediation/cfg-round1 (4 commits) fetched and independently verified: CFG01 (govulncheck 7->2), CFG06 (prints removed), CFG12 (chain-config), CFG26 (bootnode pruned). Verified in source; deployment to the fleet pending. |
| Remaining before an unconditional pass | Deploy the round-1 fixes to the live fleet (CFG Ninja to issue a deployment-verification addendum); confirm the corrected due-diligence text reached BloFin; stage-gated items (CORE-CFG08, CFG24, CFG14/16) on the pre-external-operator roadmap; one open low finding (CORE-CFG09). |
| Overall | CONDITIONAL PASS (final determination) — no open Critical or High; round-1 remediation complete and verified in source. Live deployment and BloFin delivery are the remaining near-term conditions. Not a pass of the current live code until the deployment-verification addendum is issued. |

## Code maturity by area

CFG Ninja does not assign a numeric score to a chain-core assessment. A single number implies a precision that an L1 review cannot support, and it invites exactly the misuse this report is trying to prevent — a figure quoted without its findings. This matches peer practice: protocol audits are summarised qualitatively and by per-finding severity, not scored. Instead, each area of the codebase is rated for maturity on the evidence gathered.

Ratings: STRONG — sound, with no material gap found. SATISFACTORY; appropriate, with minor observations. MODERATE; functional but with gaps that should be closed. WEAK; material deficiency requiring attention before the network carries greater trust. NOT ASSESSED; outside the reach of this assessment; see section 11.

| Area | Maturity | Basis |
|---|---|---|
| Consensus configuration | MODERATE | Slashing, rewards and the validator cycle match the mainnet preset exactly (clean). Held at Moderate because ETH1_FOLLOW_DISTANCE is 10 in the PRODUCTION config, not 2048 (CORE-CFG08, corrected from an earlier revision which wrongly confined it to a branch), and because the chain runs a generator-default deposit-contract address and testnet CONFIG_NAME. |

| Area | Maturity | Basis |
|---|---|---|
| RPC & node operations | MODERATE | Public RPC namespaces verified hardened (CORE-CFG23). Held at Moderate on three manifest observations: permissive CORS/vhosts on the mesh-bound endpoint pending confirmation the mesh is unroutable, chain-level alerting not yet in place (CORE-CFG27), and checkpoint-sync trust concentrated in one endpoint (CORE-CFG09). |
| Cryptographic sourcing | SATISFACTORY | All primitives come from Cloudflare CIRCL, exact-version pinned. No hand-rolled or proprietary cryptography anywhere — the single most common and most damaging mistake in this space was avoided. Pinned version is out of date. CORE-CFG17. |
| Post-quantum posture | MODERATE | Transport is genuinely post-quantum and mandatory; the signature layer is not enforced at consensus and both primitives are pre-standard round-3 rather than FIPS-203/204. Graded separately as PQR-2 (Experimental) in section 4. CORE-CFG13, CFG14, CFG16. |
| Execution-client integrity | SATISFACTORY | The production build matches the reviewed commit, and the modification surface is now confirmed by a mechanical diff against upstream v1.16.2 (CORE-CFG02) rather than inferred: consensus/ state/sync/block-validation are stock, and the custom code is confined to the six PQC/handshake/RPC files. Round-1 dependency and hygiene fixes verified on the remediation branch. |
| Transaction handling | MODERATE | Standard paths are stock and sound. The custom type 0x75 cannot be admitted by the pool at all, and its address derivation produces a second address for the same key. Fails closed, so no unsafe state — but a shipped feature does not function. CORE-CFG04, CFG15. |
| Secrets & credential hygiene | MODERATE | The enumeration gap is closed: the client derived the seventh seed against 256 BLS indices and 20 EOAs, found no control of any live validator or funded account, and is rotating it regardless. Method was validated against the official EIP-2333 test vectors. Held at Moderate because recurrence prevention is still absent — .gitignore covers none of the devnet tooling, *.env, jwtsecret or keymanager files, and CI performs no secret scanning. CORE-CFG19. |
| Decentralisation posture | MODERATE | Fully permissioned, Krown-operated, and disclosed as such with a stated roadmap — a legitimate MVP position, transparently described. It is also the mitigation currently holding two High findings below Critical. CORE-CFG30. |
| Peer-to-peer transport | WEAK | The rewritten handshake discards the peer identity-signature result (CORE-CFG24) — a source-confirmed authentication weakness rated Medium within the source-review scope, with its exploitable severity turning on network exposure that a separate network assessment should establish. The session-secret disclosure (CORE-CFG06) is reachable by external peers given the client's disclosed open p2p port. Neither is covered by any test. Rated Weak on the two unresolved transport defects, independent of the scope question on exploitability. |
| Testing & verification | WEAK | Measured 29.4% over the modification surface, with crypto/kyber, p2p/ rlpx and internal/ethapi at exactly 0.0%. Upstream ships rlpx_test.go, buffer_test.go and api_test.go; all are absent here — diff-confirmed against upstream v1.16.2. Neither CI workflow runs the suite. CORE-CFG34. |

| Area | Maturity | Basis |
|---|---|---|
| Dependency management | MODERATE | The seven reachable vulnerabilities are remediated in source and CFG Ninja re-verified 7->2 on the remediation branch, the residual two having no upstream fix (CORE-CFG01). Held at Moderate: the fix is not yet deployed to the fleet, and CI still runs no vulnerability, static-analysis or secret scanning, so recurrence prevention is unaddressed. |
| Configuration & deployment | MODERATE | Production trunk confirmed as core-block and update-config is being retired. Production configuration is held outside the repository because it carries secrets — a legitimate reason — with a stated plan to commit manifest structure and inject secrets at deploy time. The auditability gap remains until that lands: no third party can verify what configuration operates the chain. CORE-CFG12, CFG26. |
| Documentation & disclosure accuracy | WEAK | The two client documents contradict each other, and the one sent to the exchange is the inaccurate one — on transaction signatures, on QRNG, and on NIST standardisation. This is the most consequential area for a listing decision and the cheapest to remediate. CORE-CFG10, CFG18. |
| Consensus-layer implementation | SATISFACTORY | Now assessed from the live beacon node (2026-08-03). Client confirmed Prysm v7.1.3 by direct /eth/v1/node/version query, and the full /config/spec shows slashing, reward and validator-cycle parameters matching the mainnet preset with no weakening. The one deviation, ETH1_FOLLOW_DISTANCE 10, is carried as CORE-CFG08. CORE-CFG07 resolved. |
| State management & sync | NOT ASSESSED | Not independently exercised (requires a runnable node, which the client declined for this round). The mechanical fork diff (CORE-CFG02) confirms these paths — state transition, sync, block validation, downloader — are code-identical to stock go-ethereum v1.16.2, so residual risk is bounded and assessed low, but the runtime behaviour was not tested and is not certified. |
| Genesis & supply integrity | SATISFACTORY | Assessed from the production genesis and independently verified: CFG Ninja confirmed the genesis hash against the live chain on 2026-08-03 (exact match to the value given to BloFin), with no hidden mint and no preloaded code beyond the deposit contract. Held out of Strong only by the single-key concentration of supply and validator-admission control (CORE-CFG28), which the client acknowledges and ties to the decentralisation roadmap. |

CONDITIONAL PASS

FINAL DETERMINATION: CONDITIONAL PASS. Within the agreed scope, the assessment of Krown Network's chain core, consensus and post-quantum implementation is complete, and it identifies no open Critical or High severity finding. Of 24 findings, 10 are resolved and CFG Ninja-verified — including the round-1 remediation branch, which CFG Ninja fetched and verified directly (dependency bumps with govulncheck re-confirmed 7 to 2, both debug prints removed, the chain-config object committed, the stale bootnode pruned) and the mechanical fork diff confirming the consensus/state/sync surface is stock go-ethereum v1.16.2. On that basis CFG Ninja issues a CONDITIONAL PASS.

The conditions, and the reason the pass is conditional rather than unconditional, are stated plainly so the determination cannot be misread. Most importantly: CORE-CFG01 and CORE-CFG06 are fixed and verified in the codebase but NOT yet deployed — the live production fleet still runs commit dd105cbf, so until the rebuilt image is rolled out the fixes are not in effect on the chain, and production nodes continue to write the Kyber session secret to local logs (CORE-CFG06, a bounded exposure). The conditional pass is therefore NOT effective on the live network until: (1) the fixed build is deployed to the fleet and CFG Ninja re-verifies it live, together with the CORE-CFG06 log purge — CFG Ninja will issue a short deployment-verification addendum on that; and (2) the corrected due-diligence text is confirmed delivered to BloFin. Stage-gated items — CORE-CFG08, CFG24, and the post-quantum enforcement work — remain on the client's pre-external-operator roadmap and are disclosed, not deferred; one low-severity finding (CORE-CFG09) is open.

Scope limitation, disclosed: state management and sync runtime behaviour were not independently exercised, as the client declined to provide a runnable node. The mechanical fork diff confirms those paths are code-identical to stock go-ethereum v1.16.2, so residual risk is assessed low, but the runtime behaviour is not certified.

This is a conditional pass, not an unconditional one, and specifically not a pass of the current live code. It must not be represented as a plain "pass", quoted without its conditions, or relied upon as certifying the deployed network until the deployment-verification addendum is issued. Reproduction must be complete and unaltered. Subject to those conditions and that limitation, and on the assessed and verified surface, CFG Ninja's determination is a conditional pass.

## Conditions for a passing final report

The conditional pass above is contingent on the following. CFG Ninja independently re-verifies each before the final report; a condition is not met by a commitment to it.

**Near-term (gate the final report):**

- DONE, verified in source (branch remediation/cfg-round1): CORE-CFG12 and CORE-CFG26 resolved (chain-config object committed, production manifests committed, stale bootnode pruned); CORE-CFG01 and CORE-CFG06 fixed and independently verified (govulncheck 7->2; both debug prints removed).
- REMAINING — DEPLOYMENT: the fixed build must reach the production fleet (live mainnet still runs dd105cbf). CFG Ninja re-verifies the live build and, for CORE-CFG06, that the session-secret log purge is complete. This is what converts CORE-CFG01/CFG06 from "resolved — deployment pending" to closed on the live network.
- REMAINING — CORE-CFG18: confirmation the corrected due-diligence text has been delivered to BloFin (wording already CFG Ninja-verified).
- DONE: genesis hash independently CFG Ninja-verified against the live chain (exact match).

**Stage-gated (required before external validators; disclosed, not deferred):**

- CORE-CFG08 — raise production ETH1_FOLLOW_DISTANCE to 2048; a coordinated fleet-wide consensus-parameter change, sequenced with the pre-external-operator work.
- CORE-CFG24 — restore proof-of-possession in the RLPx handshake; before external validators.
- CORE-CFG14 / CFG16 — consensus-level post-quantum signature enforcement and hybrid ML-KEM migration; decentralisation roadmap.

**Scope caveat:**

State management and sync were not independently exercised (the client declined to provide a runnable node for this round, and accepts the caveat). This residual is now bounded by direct evidence rather than assumption: the mechanical fork diff (CORE-CFG02) confirms the state-transition, sync, block-validation and downloader paths are code-identical to stock go-ethereum v1.16.2, so the unassessed behaviour is upstream behaviour, not fork-specific logic. Residual risk is assessed low on that basis, and is disclosed rather than certified.

This assessment deliberately carries no numeric score. Peer protocol-audit methodologies summarise an L1 review qualitatively and by per-finding severity rather than by a single figure, and CFG Ninja follows that practice for the chain-core flow. The 0–100 score used in our smart-contract reports is not applied here. Severity counts, per-finding CVSS and BVSS, the maturity matrix above and the Post-Quantum Readiness Grade in section 4 together carry the assessment — and unlike a score, each of them points at the evidence behind it.

# Appendix A — Reviewed Artifacts & Addresses

| | |
|---|---|
| Repository | https://github.com/Krown-Network/Node |
| Reviewed branch | core-block |
| Reviewed commit | dd105cbf6d7ab0ceeba955fe489f92701bd430bd |
| origin/main | f494a1f |
| origin/dev | 92ca8ce |
| origin/dev-test | f302c76 |
| origin/staging | 204be60 |
| origin/dilit-multi-sig | 3acb2df |
| origin/docker-k8s-config | 92ef598 |
| origin/update-config | 4f6430b |
| Total commits in history | 169 (squashed re-import; not incremental fork lineage) |
| Upstream baseline | go-ethereum v1.16.2 stable |
| Go toolchain | go 1.23.0 |
| PQC library | github.com/cloudflare/circl v1.6.1 |
| Deposit contract referenced | 0x00000000219ab540356cBB839Cbe05303d7705Fa — standard public Ethereum mainnet constant, present in every geth codebase; not a Krown-specific address |
| Genesis premine | Empty object: no premine allocation found in the reviewed configuration |
| Credential disclosure note | No secret values, mnemonics, private keys, passwords or JWT secrets are reproduced anywhere in this report. Committed credentials are identified by file location and truncated SHA-256 fingerprint only (CORE-CFG19); actual values can be matched by fingerprint or supplied under separate cover on request. |

# Appendix B — Disclaimer

This is CFG Ninja's FINAL report on this engagement, complete within the agreed scope. Its determination is a CONDITIONAL PASS: the conditions form part of the determination and are set out in the assessment summary. Several fixes are verified in source but not yet deployed to the live fleet; CFG Ninja will issue a short deployment-verification addendum once it confirms the fixed build is live on the network. One area (state-management and sync runtime behaviour) was not independently exercised and is disclosed as a scope limitation.

This assessment is a point-in-time review of the source code identified above, at the commit identified above, and of no other artefact. It does not extend to any subsequent commit, to code deployed on any live network, to smart contracts deployed on the network, to the operating entity's cloud, personnel or physical security, or to any third-party infrastructure. Findings marked as inference rather than demonstration are labelled as such in the finding itself and must not be read as confirmed exploits; conversely, the absence of a finding is not evidence of the absence of a vulnerability. No code was compiled or executed during this review, and all repository access was strictly read-only.

A security assessment cannot prove the absence of vulnerabilities. It reduces risk; it does not eliminate it. Nothing in this document is a warranty, a guarantee, a certification, or an endorsement of Krown Network, its token, its team, or any investment in it. Nothing in this document is financial, investment, legal or tax advice. CFG Ninja accepts no liability for any loss arising from reliance on this report, and no reader should treat it as a substitute for their own due diligence.

This report records a CONDITIONAL PASS. It must not be represented as an unconditional pass, as a plain "pass", as a certification of the current live code, or quoted without its conditions; several fixes are verified in source but not yet deployed to the live fleet, state-management runtime behaviour was not independently exercised, and stage-gated items remain on the client roadmap. No numeric score is issued for a chain-core assessment, and none should be inferred or quoted. Any reproduction must be complete and unaltered; selective quotation of individual findings, of the score, or of any positive statement in isolation misrepresents the assessment and is not authorised.

The Auditor's use of AI-agentic analysis, the human oversight applied to it, the evidence standards used, and the handling of client data are disclosed in full in Appendix C. That appendix forms part of this disclaimer.

CONFIDENTIAL — prepared for Krown Network, the client, and intended by Krown for provision to BloFin in support of the exchange's listing due-diligence. Not for wider distribution without CFG Ninja's written consent.

# Appendix C — AI-Assisted Analysis, Governance & Transparency

CFG Ninja uses AI-agentic systems as part of its assessment workflow. We disclose this plainly rather than leave it to be inferred, because a client and an exchange are entitled to know how the conclusions in front of them were produced, and because the safeguards matter more than the tooling. Nothing below reduces the Auditor's accountability for this report.

**AI-Assisted Analysis.** The Auditor employs AI-agentic systems to read and cross-reference large codebases in parallel, to execute and triage deterministic security tooling, to search exhaustively for the presence and — equally important — the absence of specific patterns across every branch and commit, and to draft findings. In this engagement that included parallel review of eight branches and 169 commits, negative searches establishing that no ML-DSA, ML-KEM or QRNG implementation exists anywhere in the tree, and triage of 28 raw static-analysis results down to those actually attributable to client-written code. AI is used to increase coverage, consistency and reproducibility. It is not used to replace auditor judgement.

**Human Oversight and Accountability.** Every finding in this report was reviewed by a human auditor, verified against source, and approved before publication. Severity classification, CVSS and BVSS ratings, the code-maturity assessment, the Post-Quantum Readiness Grade and any pass or fail determination are human judgements and are not delegated to an automated system. No finding is published on the basis of an AI assertion alone. The named Auditor remains professionally accountable for the contents of this report in full.

**Deterministic Tooling.** Factual claims about dependencies, test coverage, static-analysis results and live network state derive from deterministic tools — govulncheck, gosec, staticcheck, the Go coverage profiler and direct JSON-RPC queries — not from model inference. The exact commands are recorded in the report so that the client or any third party can re-run them and obtain the same output. Where a claim rests on reasoning rather than tool output, the report says so.

**Evidence Standards.** Each finding carries an evidence grade recording how it was established: E1 demonstrated by proof-of-concept, E2 confirmed by tooling, E3 confirmed by direct source reading with file and line citation, or E4 inferred from source without being exercised. E4 findings are labelled as inferred in the finding body and are never presented as demonstrated. This grading exists specifically to prevent a plausible-sounding but unverified conclusion from carrying the same weight as a proven one.

**Limits of Automated and AI-Assisted Analysis.** Automated and AI-assisted analysis can produce confident but incorrect conclusions, particularly where source code alone does not reflect deployed reality. This engagement contains a concrete example: an initial finding regarding the network's chain identifier was materially wrong when drawn from the repository alone, and was corrected only after the client's own documentation and the live network were consulted. The correction is retained in the report rather than quietly removed. The Auditor mitigates this class of error through independent cross-verification of significant findings, live-system checks where access permits, and the evidence grading above. Matters that could not be verified are recorded as pending rather than asserted.

**Data Handling.** Client source code and non-public materials are processed solely to perform this engagement. They are not used to train models, are not retained beyond the Auditor's records-retention period, and are not disclosed to third parties except as necessary to deliver the engagement or as required by law. No secret material — private keys, mnemonics, credentials or passwords — is reproduced in any deliverable; where such material must be referenced, it is identified by file location and truncated cryptographic fingerprint only, and deliverables are mechanically checked for leakage before release.

**Human-in-the-Loop Governance.** The workflow is structured so that a human decision point precedes every consequential output: scope and access are agreed by the engagement lead; findings are verified against source before entering the report; the report is subject to a documented quality-assurance pass covering count reconciliation, evidence integrity and secret-leakage checks; and delivery to the client, or to any third party such as an exchange, is a separate and explicitly human-authorised step. No automated system publishes, transmits or communicates on the Auditor's behalf.